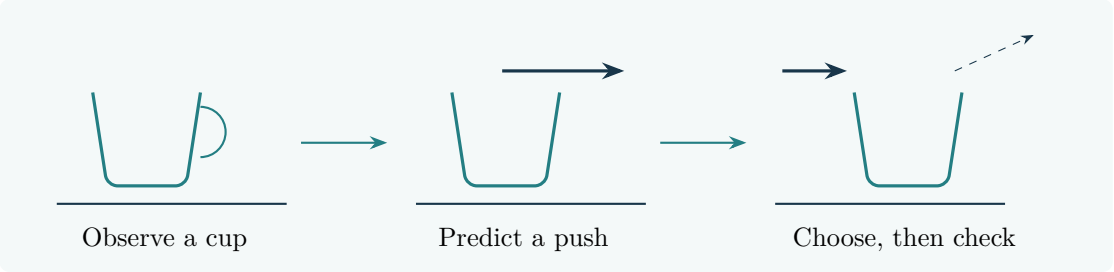
OCTOBER 2026

From Graphics to World Models

Seeing, predicting, and learning to act



A photograph shows what a scene looks like. A robot needs to know what will happen when it acts. From grasping a cup to inserting a connector, this book explores how world models can guide decisions and learning.

For students and professionals coming to robotics from computer graphics, physics, mathematics, and related fields.

Introduction

This book is for students and professionals with a background in computer graphics, physics, mathematics, or a related quantitative field who want to move into robotics, particularly world models for robot learning. It connects familiar ideas to current research and helps you choose what to study or investigate next.

Graphics gives us ways to represent scenes and form images. Physics describes how systems change under forces and constraints. Mathematics provides tools for reasoning about uncertainty, learning, and optimization. Robotics brings these ideas together around a decision: what should an agent do, given what it can observe and what it wants to achieve?

Consider a robot moving a cup onto a coaster. A good image of the cup does not tell us whether a grasp will hold. A plausible video of a successful lift does not tell us whether the proposed commands would produce it. To choose useful actions, the robot needs a model that responds to those actions and preserves the consequences that matter. To learn from that model, it also needs a way to recognize when its predictions are wrong.

We begin with scene representations and the distinction between observing a state and predicting a transition. We then study policies that generate movements, demonstrations that specify tasks, and rewards that guide learning. The later chapters examine controllable prediction, contact, and the experiments needed to judge whether a model improves decisions. The final chapter develops possible learning and research directions, from a small dynamics experiment to comparisons of published methods.

You should be comfortable with linear algebra, multivariable calculus, probability, basic machine learning, and Python at roughly third-year undergraduate level or above. Equivalent preparation through professional work or self-study is welcome. No previous robotics experience is required. If the mathematical refresher in Chapter 1 is familiar, focus on how the prediction problem changes when actions enter the picture.

Follow the cup and connector examples through the chapters, work through the derivations, and use the exercises to test your understanding. For a hands-on starting point, Chapter 6 includes a small experiment that runs on a laptop. Its simple response law makes it possible to examine why a model chooses the wrong action before tackling the same question in a visual world model.

Contents

Introduction **i**

1 From Images to Predictions **1**

1.1 What graphics gives us . . . 1
1.2 State, observation, and action . . . 1
1.3 A short probability bridge . . . 2
1.4 What changes during learning? . . . 3
1.5 Feedback turns prediction into a decision problem . . . 4
1.6 Exercises . . . 4

2 Policies, Demonstrations, and Context **5**

2.1 A cup, a camera, and a decision . . . 5
2.1.1 Why feedback changes the learning problem . . . 6
2.1.2 Why one average action can be the wrong answer . . . 7
2.2 Generating continuous actions with a flow policy . . . 7
2.2.1 Why predict a velocity field? . . . 7
2.2.2 Sampling is integration, not learning . . . 8
2.3 From a demonstrated skill to generalization and experience . . . 9
2.3.1 Heterogeneous training in $\pi_{0.5}$ . . . 9
2.3.2 Why successful demonstrations are not enough . . . 10
2.4 Adapting a learned skill to a new demonstration . . . 10
2.4.1 Context adaptation is a forward computation . . . 10
2.4.2 Synthetic diversity must preserve task relationships . . . 13
2.5 Exercises . . . 14

3 World Models, Rewards, and Policy Improvement **15**

3.1 Imagining consequences with an action-conditioned world model . . . 15
3.1.1 The interface is part of the model . . . 15
3.1.2 Why multiple views and memory matter . . . 16
3.1.3 Two uses of prediction: choosing now and learning for later . . . 16
3.1.4 What an imagined rollout actually contains . . . 17
3.2 From outcomes to rewards, values, and advantages . . . 18
3.2.1 The score is a modeling choice . . . 18
3.2.2 How a critic learns and why a baseline helps . . . 19
3.2.3 A first use of advantages: learning from experience with RECAP . . . 21

3.2.4 Likelihood is not reward . . . 21
3.3 Why a policy gradient can learn without differentiating physics . . . 22
3.3.1 The direction we want before the derivative we need . . . 22
3.3.2 A gentle log-derivative argument . . . 22
3.3.3 Old and current policies in PPO . . . 23
3.4 Flow Policy Optimization: using a loss difference as a surrogate . . . 24
3.4.1 Score a collected action with the flow model . . . 24
3.4.2 Why the ratio is a surrogate . . . 25
3.4.3 Where the gradients go . . . 25
3.4.4 Online is not synonymous with on-policy . . . 26
3.5 Two published ways to learn from imagined experience . . . 26
3.5.1 RISE: a fixed learned environment during policy improvement . . . 26
3.5.2 VLAW: refresh the simulator with real successes and failures . . . 27
3.6 The false-success trap and the evidence that breaks it . . . 28
3.6.1 An impossible grasp can look successful . . . 28
3.6.2 The VLAW reward audit: keep the denominator . . . 29
3.6.3 Correct the component for which evidence exists . . . 29
3.7 Putting the learning loop together . . . 30
3.8 Exercises . . . 31

4 From Video Prediction to a Controllable Manipulation Model 32
4.1 First decide what question the model answers . . . 32
4.2 An action is a physical contract . . . 33
4.2.1 Put observations on the correct side of the action . . . 33
4.3 Learn from future frames without needing them at deployment . . . 34
4.4 Compress what the next decision needs . . . 35
4.4.1 History and multiple views solve different ambiguities . . . 36
4.4.2 Measure the entire cycle . . . 36
4.5 Make contact error visible . . . 37
4.6 Turn the model into a reproducible data contract . . . 38
4.7 Exercises . . . 39

5 From Imagined Experience to Evidence of Improvement 40
5.1 A predictor becomes an environment through a loop . . . 40
5.2 Learn the world, learn the task, then improve behavior . . . 40
5.3 Build a target before taking a gradient . . . 41
5.3.1 Freeze the right objects . . . 42
5.4 Choose an optimizer without merging different algorithms . . . 43
5.5 When the model is wrong, which signal should change? . . . 43
5.5.1 Keep uncertainty attached to the thing it describes . . . 45

5.6 Prediction, decision quality, and improvement need different tests 45
5.6.1 Test the distinction the controller cares about 46
5.6.2 Use denominators that preserve the failure 46
5.7 One proposed iteration, from evidence back to evidence 47
5.8 Exercises 47

6 Choosing Your First Research Direction **48**
6.1 A complete experiment on a laptop 48
6.1.1 Prediction and decision metrics 49
6.1.2 Read the failure before changing the model 49
6.2 Match a direction to a question 50
6.3 Move from a toy result to a paper-level experiment 51
6.3.1 Representation is a hypothesis about sufficient information 51
6.3.2 Context must explain either intention or physical response 51
6.3.3 A learning-loop comparison needs matched evidence 51
6.3.4 Read a figure as an experimental specification 52
6.4 A two-week first pass 52
6.5 Exercises 52

A Flow Losses and Likelihood Bounds **53**

B Selected Exercise Solutions **55**

Symbols and Conventions **57**

Bibliography **62**

Chapter 1

From Images to Predictions

Imagine a cup beside a coaster. You can render a convincing picture of the scene from a new viewpoint. Can you now predict whether a robot will move the cup onto the coaster? The picture alone does not answer that question. The fingers may miss, the cup may slip, or the command may mean something different from what the model expects. To reason about these possibilities, we must connect appearance to action and action to consequence.

This chapter builds that connection. The key distinction is simple: a representation describes something; a transition model describes how it changes. A decision then requires a third ingredient: a criterion for choosing among possible changes.

1.1 What graphics gives us

Rendering starts with a description of a scene and computes an image from a chosen camera. The description may include geometry, materials, lighting, and camera parameters. Physically based rendering models the interaction of light with the scene to explain image formation (Pharr et al., 2023). It can give us controlled changes in viewpoint or illumination. It does not, merely by forming an image, determine how an object moves when touched.

Reconstruction runs part of this reasoning in the other direction: observations constrain a scene representation. NeRF represents a scene using a function of spatial position and viewing direction; density and view-dependent radiance are integrated along camera rays to synthesize images (Mildenhall et al., 2020). 3D Gaussian Splatting uses optimized three-dimensional Gaussian primitives and a visibility-aware rendering procedure (Kerbl et al., 2023). Both are useful references for novel-view synthesis. Their basic static-scene formulations do not by themselves supply an action-conditioned contact model.

The practical lesson is not that appearance is unimportant. Accurate geometry and rendering can expose alignment, generate observations, and support learning. The lesson is to ask which question a representation was trained to answer. A new-view image asks, "What would this scene look like from there?" A manipulation prediction asks, "What would this action do here?" Changing the camera and moving the cup are different interventions.

1.2 State, observation, and action

Let t be an integer control-step index. Let s_t denote the full physical state at that step: positions, velocities, and any other variables needed by our model of the system. Let o_t denote the observation available to the robot, such as an image and joint measurements. Let a_t denote one action, a command interpreted by a specified controller. These quantities have different roles even when a simple experiment stores each as a single number.

For a sliding object, position alone may not be a sufficient state: the same position with different velocity can lead to different futures. For a gripped cup, a photograph may not reveal friction. We choose a state description to make the next-state distribution depend on the current state and

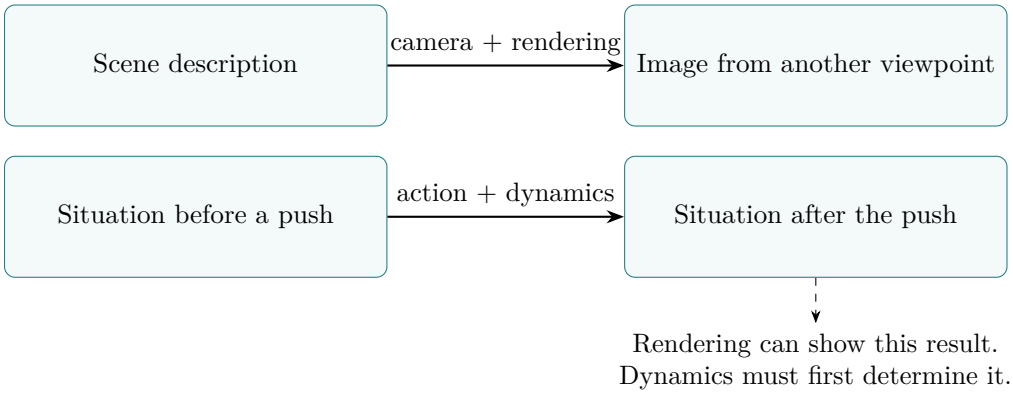


Figure 1.1: Two changes that a plausible picture can hide. Novel-view synthesis changes the viewing condition. Dynamics predicts a change in the physical situation. The two functions can be combined, but one does not establish the other.

action rather than the entire past. This is the Markov modeling assumption. An observation need not satisfy it (Sutton and Barto, 2018).

Let $\mathcal{O}$ be an observation map, $F_{\Delta t}$ a transition over a physical interval Δt, and κ the relevant environment and controller settings. In an ideal deterministic example,

$$o_t = \mathcal{O}(s_t), \qquad s_{t+1} = F_{\Delta t}(s_t, a_t; \kappa). \tag{1.1}$$

The settings might include a surface property or controller gain. Real observations and transitions are uncertain; later chapters extend this abstraction. For now, it tells us where each question belongs. A camera model specifies $\mathcal{O}$. A dynamics model approximates $F_{\Delta t}$. A policy chooses a_t.

Worked example: A photograph cannot reveal everything

Two cups occupy the same visible pose. One is lightly held, the other firmly held. A lift command may succeed in only one case. An image predictor trained on both cases cannot reliably select the right outcome unless its inputs contain distinguishing evidence. It may need tactile measurements, recent motion, or a distribution over possible futures. A larger network does not supply an unmeasured fact.

We write the available observation–action history as $h_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t)$. A short sequence can reveal motion that a single image hides. The task specification is g, for example "place the cup on the coaster." Optional demonstration context is C. This context describes the intended behavior; it need not contain every physical condition that determines success.

Checkpoint — Name the missing fact. A generated video shows a cup rising with the gripper. Identify one visible fact and one hidden fact needed to decide whether that outcome is physically supported by the action.

1.3 A short probability bridge

A model often cannot predict a unique next observation. Probability lets it represent alternatives instead of pretending they are known. Use Pr for the probability of an event and a vertical bar for conditioning. For example, $\Pr(\text{slip} \mid h_t, a_t)$ is the probability of a slip given the available history and proposed action. Conditioning restricts the information being used; it does not guarantee that the estimate is correct.

Suppose a model assigns probability 0.7 to a secure lift and 0.3 to a slip. If a secure lift receives reward 1 and a slip reward 0, the model's expected reward is $0.7 \times 1 + 0.3 \times 0 = 0.7$. Expectation, written $\mathbb{E}$, is an average weighted by probabilities. It is not a promise that any one trial achieves the average. Later we average accumulated rewards over entire futures.

A continuous action distribution uses a *density*. The probability of one exact real-valued command is generally zero; intervals have probability obtained by integrating the density. A density value can exceed one without violating probability rules. For example, a uniform distribution over an interval of width 0.1 has density 10 on that interval, yet total probability one. This distinction matters when later chapters compare action densities.

Let π_θ be a policy distribution with parameters θ, and p_ϕ a world-model distribution with parameters ϕ. The notation

$$a_t \sim \pi_\theta(\cdot \mid h_t, g, C), \qquad \widehat{o}_{t+1} \sim p_\phi(\cdot \mid h_t, a_t) \tag{1.2}$$

means that we sample an action, then sample a predicted next observation. The dot stands for the variable being sampled; a hat marks a prediction or estimate. The first distribution answers what to do. The second answers what might follow. Its accuracy must be checked against a measured o_{t+1}.

One subtlety matters. A conditional model learned from recordings can fit correlations that fail when a different policy chooses actions. Perhaps the demonstrations contain large pushes only when the object is far from an obstacle. The model may have little evidence about a large push near the obstacle. Writing an action after the conditioning bar does not establish reliable prediction for every intervention. We must test action coverage and the availability of relevant state information.

1.4 What changes during learning?

A model contains adjustable numbers called parameters. Training chooses them to reduce a loss on examples. Let $\mathcal{D}$ be a dataset, i index one example, and N be the number of examples in a batch. A loss $\mathcal{L}(\theta)$ measures how well a policy with parameters θ fits the selected training objective. Let $\nabla_\theta \mathcal{L}$ be its gradient, the vector of local derivatives with respect to those parameters. A gradient-descent step with positive learning rate η_{learn} is

$$\theta_{\text{new}} = \theta - \eta_{\text{learn}} \nabla_\theta \mathcal{L}(\theta). \tag{1.3}$$

The minus sign moves against the direction of increasing loss. A finite step can still fail to reduce the loss if it is too large. Even a lower training loss need not mean better behavior outside the training distribution.

Worked example: One parameter, one update

For this example only, let a scalar policy parameter θ be a constant normalized action and let the demonstrated target be 0.8. Choose $\mathcal{L}(\theta) = (\theta - 0.8)^2$. At $\theta = 0.2$, the derivative is $2(0.2 - 0.8) = -1.2$. With $\eta_{\text{learn}} = 0.1$, the updated value is 0.32. The squared error drops from 0.36 to 0.2304. This demonstrates fitting a target, not proving that the target is safe or successful.

Inference evaluates the model with fixed parameters. The output can change because the image, task, or demonstration changed. *Training* changes the parameters. A robot correcting its grip after receiving a new image may be using ordinary feedback with no weight update. Chapter 2 will add another distinction: the internal numerical steps that generate an action are also not parameter updates.

Separate training, development, and test data by independent episodes or conditions appropriate to the claim. Adjacent frames from the same attempt are strongly related. Putting almost identical frames on both sides of a split can make generalization appear easier than it is. Use development data to make design choices and reserve the test set for evaluation. If repeated test inspection drives model changes, that test set has become part of development.

1.5 Feedback turns prediction into a decision problem

Suppose a robot chooses a long sequence of commands from one initial photograph and never observes again. A small initial error can persist through the sequence. In feedback control, it observes after acting and chooses again from the updated evidence. A learned policy is one way to implement this mapping; feedback itself does not require a neural network.

A world model adds another option: compare a few candidate actions before execution. The robot might imagine pushing, retracting, and moving sideways, score their consequences, and execute one. This is model-based planning. Alternatively, it might train a policy using many imagined attempts so that later choices require less search. This is learning from imagined experience. They have different computational costs and different failure modes.

A predictor can be useful even when imperfect if it preserves the distinction relevant to the choice. It can be useless despite attractive images if it predicts success for every action. We therefore need three increasingly demanding questions:

1. Does it predict a recorded action's consequence on held-out data?
2. Does it help select a better action among alternatives with independent outcome evidence?
3. Does the resulting policy improve behavior when it acts, observes, and acts again?

The rest of the book follows this sequence. First we need a policy expressive enough to produce useful movements. Then we need feedback that distinguishes better from worse experience. Finally we need an experiment that could show our proposed improvement is wrong.

1.6 Exercises

Exercise 1.1. Two images show a cup at the same position, but one cup is moving. Why can position alone fail to be a Markov state? Name an additional measurement that could help.

Exercise 1.2. Repeat the scalar update above with learning rate 0.5. What is the new parameter? Why does this one-example result not prove generalization?

Chapter 2

Policies, Demonstrations, and Context

A robot first needs to imitate a demonstrated motion. Then it must handle a changed scene and interpret a new demonstration. This chapter follows those steps through the same cup-moving problem. The papers enter when their mechanisms answer a concrete question about action generation or context. Chapter 3 then asks how the robot can learn from an action's consequences.

Our destination is a training loop, but this chapter starts with action generation. We will learn how a policy generates a continuous action and how a demonstration can condition that action. Chapter 3 will then introduce feedback that changes the policy weights. Keep one question in mind throughout: *what information would the robot need to do better on its next attempt?*

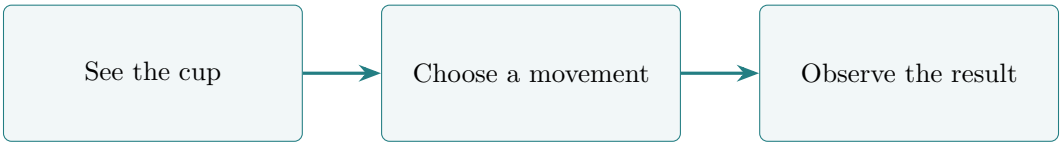


Figure 2.1: One task, changing evidence. Feedback lets the next movement respond to a shifted cup. The complete policy, rather than object recognition alone, must choose commands (Physical Intelligence, 2024).

See Physical Intelligence (2024), Figure 1.

2.1 A cup, a camera, and a decision

A cup stands on a table. A person demonstrates lifting it and placing it on a coaster. On the next attempt the cup is slightly farther away, its handle faces another direction, and the wrist camera sees the rim only after the gripper begins to move. A useful robot must preserve the intention while changing the motion. Replaying the demonstrator's joint commands would generally miss the cup. Recognizing the word "cup" would not tell the fingers when to close. The problem is to select actions from incomplete, changing evidence.

We begin by distinguishing the physical situation from what the robot can observe. Let s_t denote the full physical state at control step t, where t is an integer index: robot configuration, cup pose, velocities, contact conditions, and other relevant variables. An observation o_t might contain external and wrist-camera images together with joint angles and gripper state. It usually does not reveal every component of s_t. For instance, a camera may show fingers around a cup without revealing whether friction is sufficient to hold it. Calling an image a "state" is convenient shorthand in many papers, but it does not make the physical system fully observed.

An action a_t is a command in a defined interface. It might specify joint velocities, a desired joint configuration, or a relative end-effector pose and a gripper command. These are different mathematical objects. A translation of 1 could mean a metre, a normalized coordinate, or a displacement in another frame. Consequently, the first engineering question is what a vector means, at what rate it is executed, and which controller interprets it. A learning algorithm cannot repair an unspecified action convention.

2.1.1 Why feedback changes the learning problem

A photograph of the cup is an input, but the robot's next input depends on what it does now. Moving the wrist changes the camera view; touching the rim can shift the cup; closing the fingers can establish or miss a grasp. The interaction therefore forms a feedback loop: observe, act, receive a new observation, and act again. A policy participates in creating the future situations in which it will be evaluated. This is a central difference from predicting labels on a fixed set of images (Ross et al., 2011).

History matters because two nearly identical images can require different actions. A stationary cup and a slowly slipping cup may look similar in one frame. A short sequence can reveal the motion. We write $h_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t)$ for the available observation–action history through step t. A model may compress this history or use only a recent window; those are approximations to the information represented by h_t. The *Markov* idealization says that a state contains enough information to predict the next-state distribution given an action, without needing earlier history. A camera image need not meet that idealization. Using history is one way to address missing information, although a finite learned summary is not guaranteed to retain everything (OpenAI, 2018c).

A *policy* chooses an action from available information. Let g specify the task, C denote optional demonstration context, and θ denote the policy parameters. A stochastic policy defines a conditional action distribution:

$$a_t \sim \pi_\theta(\cdot \mid h_t, g, C), \tag{2.1}$$

The vertical bar means "given," and $\sim$ means "sampled from." The policy parameters are its network weights. Sampling selects one action from the conditional distribution. Different samples can yield different sensible approaches to the same cup. A deterministic controller is a useful limiting case, but it should not be silently substituted for a stochastic policy in likelihood-based derivations.

The network does not contain a separate weight for every possible grasp. Its weights encode a shared computation that responds to the input. During *inference*, we run that computation with the current inputs and fixed weights. During *training*, an objective measures some discrepancy or preference, and an optimizer changes the weights. A policy can therefore react continuously to new images without continuously learning new weights.

These two processes occur at different scales. Within one cup attempt, feedback can correct a motion because the image changed. Across many attempts, training can make that correction more likely because the weights changed. A successful recovery during one trial does not, by itself, show that anything was retained for the next trial.

A demonstration is a recorded trajectory: observations aligned with the commands that produced subsequent observations. A collection of such trajectories supports *imitation learning*, in which the policy learns to generate actions resembling the demonstrated behavior under the corresponding conditions. The supervision is richer than a final image of a cup on a coaster: it contains decisions made during approach, contact, lift, transport, and release. Alignment matters. Pairing the image after a successful grasp with the command before contact teaches a different relationship from pairing the command with its actual input observation.

A manipulation policy can propose several actions at once. Let H be the number of actions, d_a the dimension of each action, and $K \leq H$ the number executed before replanning. Use $\mathbb{R}^{H \times d_a}$ for real arrays of that shape. The proposed *action chunk* is

$$\mathbf{a}_t = (a_t, a_{t+1}, \ldots, a_{t+H-1}) \in \mathbb{R}^{H \times d_a}. \tag{2.2}$$

Bold $\mathbf{a}_t$ denotes the whole chunk; a_t denotes one action. A policy density evaluated on a chunk is a joint density over that chunk, not a product of independent action densities unless explicitly

assumed. The entries are proposed future actions, indexed by their intended execution steps. The controller executes the first K actions, observes again, and replans; the remaining proposals may therefore be replaced. This receding-horizon pattern combines temporally coherent motion with feedback. A cup that slips after contact should alter the next decision even if the previous chunk confidently predicted a lift. Action chunking is central to Diffusion Policy and the flow policies considered here (Chi et al., 2024; Physical Intelligence, 2024).

Worked example: Three clocks that must stay separate

Imagine a toy controller that predicts eight actions, executes two at a time, and uses six numerical integration steps to generate each chunk. Eight is a prediction horizon, two is an execution prefix, and six is a sampler budget. None specifies how many optimizer updates occur. If the control period is 0.05 seconds in this example, the executed prefix covers 0.1 physical seconds; the six sampler steps need not correspond to any physical interval. Training could be completely disabled during deployment. A trace should therefore record physical timestamps, sampler settings, and model versions separately.

2.1.2 Why one average action can be the wrong answer

Imagine that the cup has two accessible grasp regions, one on each side of an obstacle. Half the demonstrations approach from the left and half from the right. If a deterministic regressor minimizes squared error against both motions under indistinguishable inputs, it is encouraged toward their conditional average. That average can pass through the obstacle even though neither demonstrated approach does. This illustrates *multimodality*: several separated families of actions can be valid. The value of expressive action distributions is a motivation for generative policies such as Diffusion Policy (Chi et al., 2024).

A distribution can retain both alternatives and sample one coherent approach. The random seed selects variation; the observation and task determine which variation is appropriate. Randomness alone is not competence: sampling from an untrained distribution produces arbitrary commands. Training must shape the random input into useful motions. This gives us a precise question for the next section: how can we learn a flexible distribution over an entire continuous action chunk?

2.2 Generating continuous actions with a flow policy

2.2.1 Why predict a velocity field?

The role of a flow policy is to turn a simple random input into a structured action chunk conditioned on the scene and task. In π_0, a pretrained vision–language backbone provides representations and a continuous action expert is trained with conditional flow matching (Physical Intelligence, 2024). This connects semantic understanding to motor output without requiring every real-valued action coordinate to be generated as an ordinary word token.

The learning construction is especially simple when we already have a target action chunk $\mathbf{a}$. Draw standard Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ of the same shape, where I is the identity covariance, and sample artificial flow time τ uniformly from $[0, 1]$ for this basic construction. Here we use the convention that $\tau = 0$ is noise and $\tau = 1$ is data; papers or implementations may reverse it. Let X_τ be a noisy intermediate chunk and u_τ its reference velocity. Define a straight training path

$$X_\tau = (1 - \tau)\epsilon + \tau\mathbf{a}, \qquad u_\tau = \frac{dX_\tau}{d\tau} = \mathbf{a} - \epsilon. \tag{2.3}$$

The target velocity u_τ is known because both endpoints were selected during training. It is not measured from the robot and is not an estimated reward. Write $c_t = (h_t, g, C)$ for the policy's conditioning information, and omit the subscript when discussing a generic training example. The velocity network $v_\theta(X_\tau, \tau; c)$ predicts the reference velocity. Let $\mathbb{E}$ denote expectation over the specified random inputs and $\|\cdot\|$ the Euclidean norm after flattening the chunk. Denote the conditional flow-matching (CFM) loss by $\mathcal{L}_{\text{CFM}}$:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{(c,\mathbf{a}),\epsilon,\tau}\left[\left\|v_\theta(X_\tau, \tau; c) - (\mathbf{a} - \epsilon)\right\|^2\right]. \tag{2.4}$$

The expectation averages over training chunks, noise, and flow time. Different time-sampling distributions, weights, and normalization conventions change the practical objective; they will matter again in Flow Policy Optimization (FPO). For now, its purpose is supervised velocity prediction (Lipman et al., 2024).

Training uses known actions to construct the local prediction problems that will later let the model generate new actions. For every demonstration chunk, we can manufacture many noisy intermediate points and their target velocities. The network learns from these local questions instead of being asked to discover an entire successful sampling trajectory in one training step. Conditional flow matching supplies this tractable regression construction (Lipman et al., 2023).

Why begin with Gaussian noise? It provides a distribution that is easy to sample and has no task-specific structure. The network learns the transformation that supplies the structure. A noise tensor here has the shape of an action chunk; it is a computational starting point, not a command sent to the robot. Only the completed sample is interpreted as an action proposal.

A point X_τ can be compatible with many target chunks. The network therefore does not learn a separately labeled straight line for every future inference sample. Squared-error regression learns a conditional average velocity at each noisy location and conditioning input. Integrating this learned field transports a distribution of noise samples toward a distribution of actions. The *training reference paths* are straight, while the *learned sampling paths* need not be.

This also resolves an apparent conflict with the earlier warning about averages. A deterministic action regressor averages *final actions given the observation.* The flow network averages *local velocities given the observation, noisy location, and flow time.* Those additional inputs distinguish where a sample currently lies. Different noise samples can travel into different action modes rather than all being mapped to one average endpoint. The distinction between a conditional reference field and the resulting marginal field is part of the flow-matching construction (Lipman et al., 2023, 2024).

Worked example: One coordinate of a cup approach

For this scalar example, let a be one normalized target-action coordinate. Suppose its value is $a = 0.8$, the sampled noise is $\epsilon = -0.4$, and $\tau = 0.25$. Then $X_\tau = -0.1$ and the known reference velocity is 1.2. If the network predicts 0.9, its squared error for this coordinate is $(0.9 - 1.2)^2 = 0.09$. Gradient descent adjusts θ to reduce such errors over many examples. This number says nothing about whether the cup was moved successfully. It evaluates a prediction on an artificially noised action, not a physical task outcome.

2.2.2 Sampling is integration, not learning

At inference there is no known expert endpoint $\mathbf{a}$. Start from $X_0 \sim \mathcal{N}(0, I)$ and solve

$$\frac{dX_\tau}{d\tau} = v_\theta(X_\tau, \tau; c), \qquad \mathbf{a}_t = X_1. \tag{2.5}$$

Let τ_j and τ_{j+1} be adjacent points on the sampling grid, indexed by j. An Euler step is

$$X_{\tau_{j+1}} = X_{\tau_j} + (\tau_{j+1} - \tau_j) v_\theta(X_{\tau_j}, \tau_j; c). \tag{2.6}$$

The changing variable is X_τ, while θ stays fixed throughout the solve. Numerical integration creates one candidate action chunk. An optimizer update later changes θ using a training objective; it is a different operation with a different purpose. This ordinary differential equation (ODE) evolves a sample in action space; it does not describe the robot's physical motion.

The original π_0 setup uses a 50-action chunk and ten flow integration steps (Physical Intelligence, 2024). These numbers do not mean that the robot executes ten actions, or that its weights are updated ten times. Nor does reducing the solver step size repair an incorrect learned distribution. It may reduce sampling discretization error while leaving representation error, data bias, and physical control error intact.

There is also an operational distinction between generation and execution. A sampled chunk must be unnormalized, converted into the robot's command convention, and scheduled by the controller. Saturation, latency, and replanning affect the executed trajectory. If a future world model is trained on intended commands but the hardware executes clipped commands, the model and policy can disagree even when each software module is internally consistent. The meaningful training tuple records what was actually applied.

Checkpoint — What changed?. During flow sampling, the noisy action variable changes. During execution, the physical state changes. During policy training, the parameters change. Explain which of these three changes occurs when a new Gaussian seed produces a different grasp while the checkpoint remains identical.

We now have a way to generate flexible motion. The next limitation is coverage: what makes a learned cup-moving skill useful in a different room, with another object, or under a new instruction?

2.3 From a demonstrated skill to generalization and experience

2.3.1 Heterogeneous training in $\pi_{0.5}$

A robot needs several kinds of knowledge at once. It must identify the requested object, infer a useful subtask, and execute a precise motion. Training only on one cup trajectory cannot provide all of that. $\pi_{0.5}$ combines heterogeneous sources, including web multimodal data, data from different robots and environments, and high-level subtask supervision. Its pretraining uses FAST, an action tokenizer that compresses continuous action sequences into discrete tokens; its post-training introduces continuous flow-matching action prediction. At inference, a predicted high-level subtask conditions low-level actions (Physical Intelligence, 2025a).

For the running example, "put the cup on the coaster" might require the currently relevant subtask "grasp the cup" before "place the cup." This decomposition helps explain how broad semantic knowledge can influence a short action chunk. It is not a guarantee that the hierarchy is correct. The robot can recognize the cup but misunderstand the target coaster, or choose the correct subtask but fail to establish contact. Generalization must be located: across appearances, object configurations, tasks, environments, or embodiments. These are related but distinct tests.

The training stages also solve different representation problems. Broad vision–language training helps connect images and words; action-labeled training connects such representations to commands. A high-level subtask reduces ambiguity about which part of a long instruction should govern the next short chunk. Neither the phrase "grasp the cup" nor a good visual feature contains an executable grasp by itself. The action expert must still learn that mapping. Thinking in terms of these roles makes the move from π_0 to $\pi_{0.5}$ intelligible without memorizing a list of datasets.

Broader training data can improve coverage, but each source must supervise the right function. Web images can teach visual concepts without supplying robot commands. Robot datasets may use different action conventions or omit some measurements. Combining them requires compatible representations, explicit missing-data masks, and a clear account of which examples supervise language and which supervise action.

2.3.2 Why successful demonstrations are not enough

Suppose the pretrained robot closes its fingers slightly early. The demonstration dataset mostly contains centered grasps, so its next observation is unfamiliar: the cup has moved, but it is not held. Continuing to imitate a clean lift from that observation may make the failure worse. The robot needs information about its own mistakes and recoveries, not only what competent behavior looked like on the expert's distribution.

The difficulty is not merely that the dataset is small. The learner's own choices change the distribution of observations it encounters. A slightly off-center approach creates an off-center grasp image; the next imperfect action creates an even less familiar situation. This feedback can compound errors. The DAgger work formalizes this issue and studies collecting expert labels on states visited by the learner (Ross et al., 2011). We use it here to explain the problem, not to claim that all the methods in this chapter implement DAgger.

There are two complementary ways to obtain additional guidance. A person can show what to do in a situation the policy reached. Alternatively, an outcome can tell us that a tried behavior worked better or worse than expected, even without an expert specifying every action. RECAP will provide a concrete example of combining experience and corrections once we have defined the required value and advantage signals in Section 3.2.

Before learning persistently from many attempts, however, consider an easier and more immediate request: a new user demonstrates a different destination for the cup. The robot should be able to interpret that demonstration now. This motivates changing its conditioning input while retaining the competence already learned in its weights.

2.4 Adapting a learned skill to a new demonstration

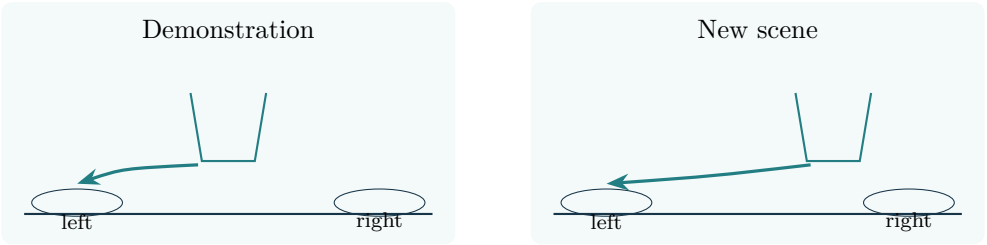


Figure 2.2: A demonstration changes the question. Changing demonstration context can change behavior without an optimizer update (Qian et al., 2026a).

2.4.1 Context adaptation is a forward computation

Let C_1 show a cup placed on a coaster and C_2 show it placed on a shelf. A context-conditioned policy can produce different actions from the same current observation:

$$\pi_\theta(\mathbf{a} \mid o, g, C_1) \neq \pi_\theta(\mathbf{a} \mid o, g, C_2). \tag{2.7}$$

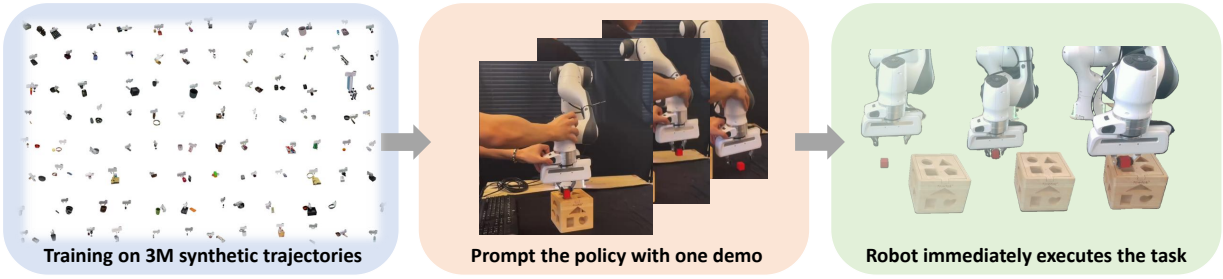


Figure 2.3: A demonstration specifies the task. Figure 1 from Qian et al. (2026a). CC BY 4.0.
Reading the paper figure. Compare the demonstrated scene with the new execution. The scene changes, while the relation specified by the demonstration should be preserved. Training the policy and conditioning it at inference are separate operations.

There is no contradiction in obtaining new behavior while keeping θ fixed. The conditioning input changed. During earlier training, the network learned how to interpret demonstrations; at inference it applies that learned computation to a new one. This is in-context adaptation, distinct from persistent parameter learning.

For that behavior to be learned, training must make context informative. Consider a training pair in which one trajectory demonstrates a task and another performs the same task from a changed configuration. The policy receives the first as context and predicts actions in the second. Across many such pairs, always following one fixed motion is inadequate: the context tells it which relationship should be preserved. In the cup example, that relationship could be "this cup goes onto that coaster," while positions and camera appearances vary. This is the role of context–task pairing in SynthICL (Qian et al., 2026a).

It is helpful to distinguish reading a demonstration from replaying it. Replaying uses the old commands at their old times. Reading extracts information that can condition a fresh action from the current observation. A new grasp may therefore differ from the demonstration while still carrying out the demonstrated task.

In SynthICL, context and current images pass through the same frozen DINOv3 visual backbone. A spatiotemporal encoder organizes demonstration features; current-image features query the encoded demonstration through cross-attention. An action decoder then produces flow velocities. A parallel subgoal-image predictor supplies auxiliary training supervision and can be disabled at inference; predicted subgoals are not explicit runtime control guidance (Qian et al., 2026a).

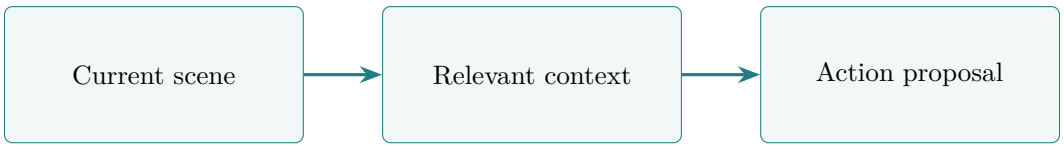


Figure 2.4: Two information sources meet. The current scene describes where the robot is; demonstration context describes the intended relation. Attention combines features from both (Qian et al., 2026a; Vaswani et al., 2017).

Cross-attention provides a concrete interpretation of "using context." Think of the current features as asking what information in the demonstration is useful now. Queries represent the question, keys provide features against which relevance is scored, and values carry the information to be combined. These are learned vectors, not literal textual questions or a hand-coded search table. Scaled dot-product attention computes compatibility scores, normalizes them with softmax, and forms a weighted sum of the values (Vaswani et al., 2017).

Let $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ be matrices of current queries, demonstration keys, and demonstration values. Let d_{key} be the shared query/key feature dimension. With $\top$ denoting transpose, scaled

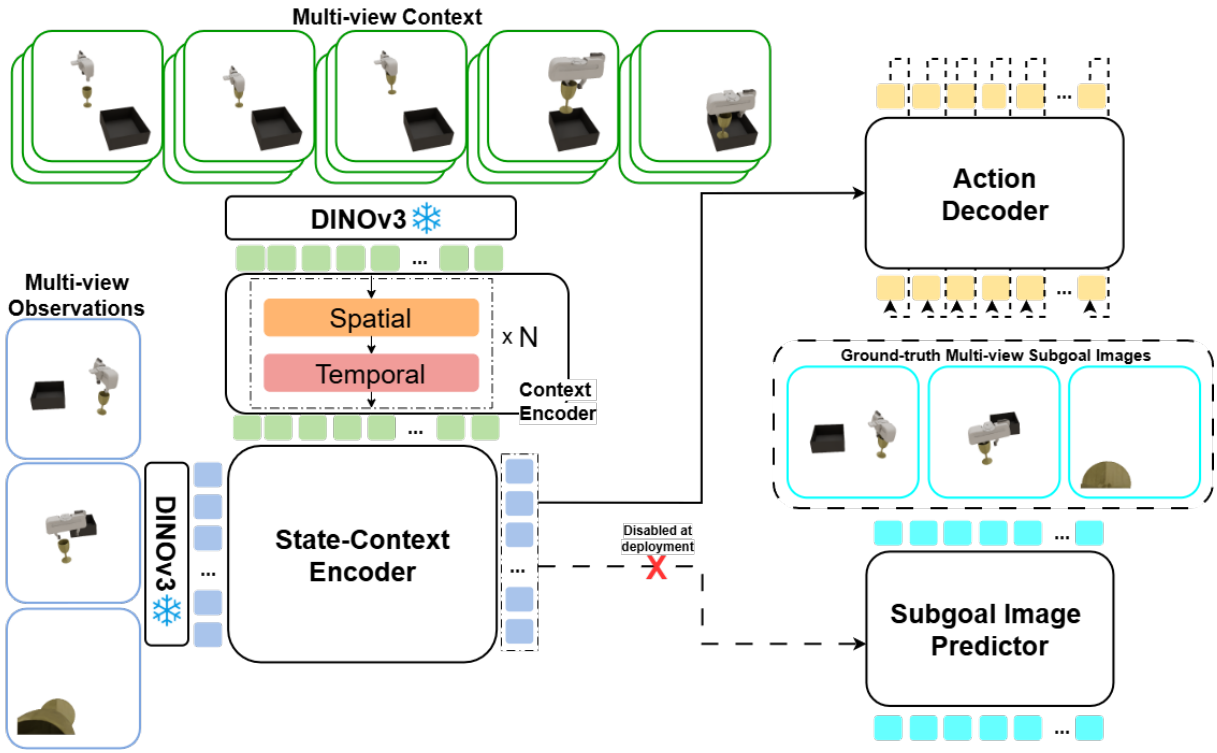


Figure 2.5: Inside the context-conditioned policy. Figure 2 from Qian et al. (2026a). CC BY 4.0.

Reading the paper figure. Follow the current and demonstration images through the shared frozen visual backbone. The auxiliary subgoal branch supplies training supervision; its predicted image is not required runtime control guidance. The separate visual boxes do not denote independently trained backbones.

dot-product attention is

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\mathrm{key}}}}\right)\mathbf{V}. \tag{2.8}$$

Softmax converts scores to nonnegative weights summing to one and is applied over the available keys for each query. For the cup, a query can emphasize the grasp region early and the destination later. Attention performs soft aggregation; it does not necessarily select a unique matching demonstration frame or recover a symbolic task program. Positional and temporal information help distinguish seeing the cup before contact from seeing it after release. Changing the demonstration changes $\mathbf{K}, \mathbf{V}$ and the resulting features even with every weight frozen.

The auxiliary subgoal task has a different role. If the training target says what the next meaningful waypoint should look like, errors in that prediction can encourage the shared representation to retain task stage and destination information. The action decoder can benefit from those features without consuming a generated subgoal image at runtime. In SynthICL this is a training aid; it does not insert a separate image planner between observation and action (Qian et al., 2026a).

The supervision for actions and the information in a context demonstration also need not be identical. SynthICL uses visual context without requiring the context's action labels at test time (Qian et al., 2026a). Thus our generic notation C must not be interpreted as saying every method consumes an observation–action demonstration pair. The target training trajectory still supplies the actions being learned. This distinction matters when designing a demonstration recorder for a user.

2.4.2 Synthetic diversity must preserve task relationships

Rendering experience offers an intuitive entry point. Varying texture and illumination can improve visual robustness, but a context-conditioned policy must also learn relationships: which object is moved, where it goes, how grasp and release divide the sequence, and how the corresponding target trajectory changes when objects move. If context and target examples are paired incorrectly, visually excellent images teach the wrong task.

SynthICL constructs synthetic context–task pairs with object-centric waypoints and attach/release rules rather than full contact simulation. Its official project reports 79% average success on 16 unseen real-world tasks with one demonstration (Qian et al., 2026b). This is evidence for transfer under that protocol; it is not a proof that synthetic attachment rules reproduce grasp physics. The conceptual lesson is that a useful training prior and a physically faithful simulator are different things.

Instant Policy takes another representational route: segmented point clouds and a graph connect demonstrations, the current observation, and future actions. Graph diffusion learns action generation from semantically consistent pseudo-demonstrations (Vosylius and Johns, 2025). Both approaches teach how to infer task structure from context, but their sensing and architecture assumptions differ. We should not transfer a result from one representation to another without checking the required inputs.

Worked example: A new destination is not a new learned checkpoint

Give the robot a demonstration that places the cup on the left coaster instead of the right. The current action distribution may change immediately. Remove that demonstration and restore the previous context: the behavior should again depend on that previous input. No knowledge has necessarily been consolidated across users. Persistent improvement would require retaining trustworthy examples, training a candidate checkpoint, and testing both the new behavior and

prior capabilities. A generated subgoal image is a prediction made by the model; it does not become an independently verified target simply because it looks useful.

Checkpoint — Two adaptation experiments. Compare changing only the demonstration with changing only the checkpoint. What evidence would isolate context adaptation, and what evidence would establish persistent learning? Keep current observations, task definitions, and evaluation conditions matched.

So far, a policy maps evidence to actions. To improve it cheaply, we would like to explore consequences before executing every candidate on a robot. Context tells the robot what behavior is intended; a world model would tell it what its actions might cause.

2.5 Exercises

Exercise 2.1. A policy proposes 12 controls, executes 3, and uses 8 sampling evaluations. The control interval is 0.04 seconds. What physical duration is executed before replanning? Which number determines the optimizer update count?

Exercise 2.2. Design a matched test that distinguishes response to a demonstration from persistent weight learning. State what is fixed and what changes.

Chapter 3

World Models, Rewards, and Policy Improvement

The policy in Chapter 2 can generate actions and read demonstrations. Now it needs to compare consequences. We first construct imagined experience, then define reward and advantage, and finally derive an update. The gradient and FPO sections develop the update at equation level. They belong to the technical argument: the later reliability discussion depends on understanding what the objective can and cannot establish.

3.1 Imagining consequences with an action-conditioned world model

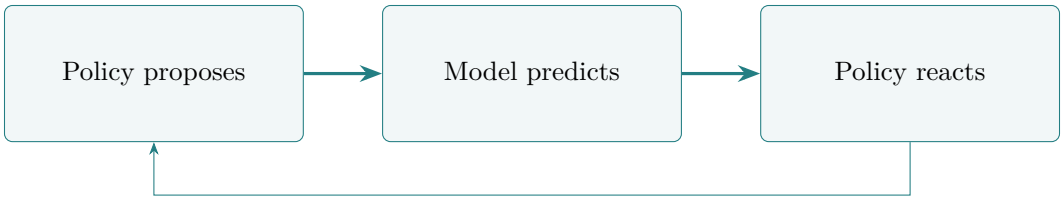


Figure 3.1: A generated future becomes an input. Following the return arrow makes this a policy-in-the-loop experiment. Prediction errors can affect later decisions (Guo et al., 2026b).

See Guo et al. (2026b), Figure 1.

3.1.1 The interface is part of the model

A learned dynamics model supplies a distribution over future observations or latent states. Let p_ϕ denote this conditional distribution, with parameters ϕ, and let a hat mark a predicted observation. For an action chunk,

$$\widehat{o}_{t+1:t+H} \sim p_\phi(\cdot \mid h_t, \mathbf{a}_t), \tag{3.1}$$

This distribution has a different role from the policy distribution: the policy proposes actions, while the world model predicts their consequences.

Both models may use flow matching, but they generate different objects. For the policy, the clean training target is an action chunk. For the world model, it is a future observation or latent trajectory. A *latent* is an encoded representation of an observation, used to reduce the size or organize the information on which prediction operates. Decoding converts it back into an image when a visual policy needs one. A shared generative technique does not make the two models interchangeable: one chooses an intervention and the other predicts what follows it. A model can generate a plausible cup-lifting video yet fail this action-conditioned task if it shows the cup lifting when the commanded gripper never touches it.

A usable interface needs a shared action convention, camera layout, time spacing, and treatment of robot state. If the policy produces relative Cartesian displacements and the world model expects

joint-space commands, an adapter is required. Its inputs, kinematic assumptions, and clipping must be explicit. Visual conditioning on an end-effector pose is not equivalent to simulating torque-driven contact. It conveys where the robot is meant to be, but contact feasibility remains something the learned dynamics must represent or external checks must constrain.

Ctrl-World adapts a pretrained video diffusion model using joint multiview prediction, frame-level pose conditioning, and sparse pose-conditioned memory. Its described adapter converts joint-velocity commands to joint configurations and then uses forward kinematics to obtain Cartesian poses (Guo et al., 2026b). These conversion steps are essential context for interpreting "action controllability."

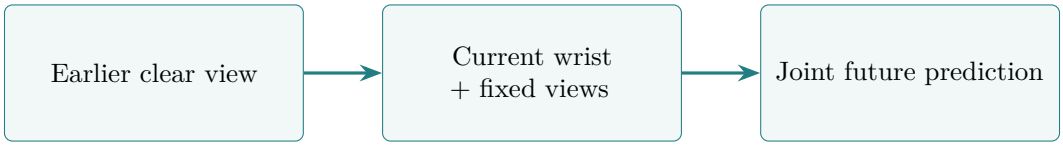


Figure 3.2: An occluded cup still has a history. A remembered view can preserve earlier geometry. It is not a new measurement of current contact (Guo et al., 2026b).

See Guo et al. (2026b), Figure 2.

3.1.2 Why multiple views and memory matter

The wrist camera can observe the grasp closely but lose sight of the destination. An external camera can show the destination while hiding the fingers behind the cup. Joint prediction lets evidence from one view constrain another. Independent plausible views are insufficient: one could show a held cup while another shows it on the table. Consistency means they describe the same underlying event, not merely that each image looks realistic.

History and memory address a related temporal ambiguity. Recent observations indicate motion; selected older observations retain information that becomes occluded. If the cup moves behind the gripper, a memory of its earlier pose may help maintain identity and scene structure. But memory cannot reveal friction or a hidden object never observed, and recycling a generated image into memory can preserve an error as if it were evidence. Keeping observed and predicted history distinguishable is therefore a proposed engineering safeguard, not an automatic property of attention.

WEAVER predicts multiview image latents and proprioception from actions, recent history, and sparse older memory. It adds latent reward and critic heads so candidate outcomes can be scored without repeatedly decoding images for a large external judge. Its reward supervision comes from RoboMeter progress labels; the critic uses bootstrapped targets (Jain et al., 2026). These heads make scoring cheaper, but do not create task truth from dynamics alone. In the inspected WEAVER version, actions are joint-position differences, with an additional velocity-to-position adapter for interfacing with other action conventions (Jain et al., 2026). This differs from Ctrl-World's Cartesian-pose conditioning. An adapter must preserve timing and command meaning; matching vector dimensions is insufficient.

3.1.3 Two uses of prediction: choosing now and learning for later

Suppose the robot proposes three cup approaches. It could predict each outcome, select the most promising candidate, execute a prefix, observe again, and repeat. That is *planning with a model*. The decision changes, but the policy's weights can remain fixed. Alternatively, it can retain many imagined attempts and use their scores to train a new policy. That is *learning from model-generated experience*. Later decisions may improve without rerunning the same search.

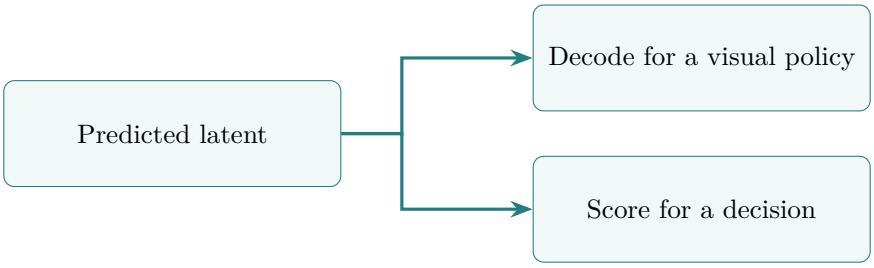


Figure 3.3: One representation, two uses. Decoding and scoring serve different consumers. A low-cost score still depends on the quality of its supervision (Jain et al., 2026).

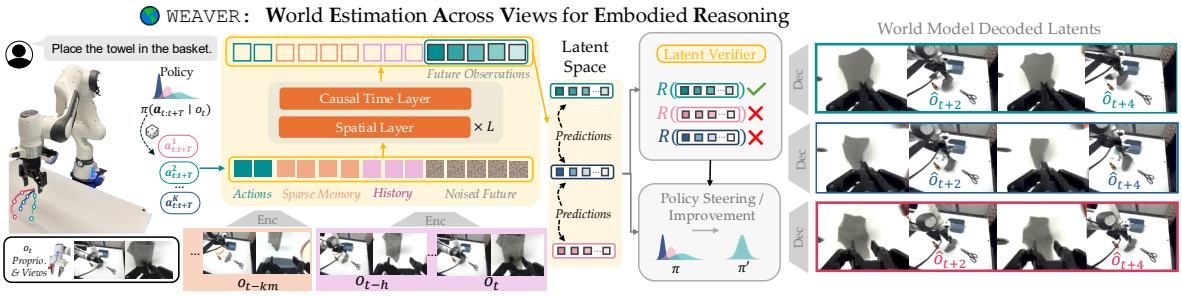


Figure 3.4: Prediction and scoring inside WEAVER. Figure 2 from Jain et al. (2026). CC BY 4.0.

Reading the paper figure. Trace the action and history inputs into latent future prediction, then distinguish image decoding from latent reward and critic heads. A cheaper score still depends on its training labels and bootstrap targets.

These are different uses of a predictive model; WEAVER investigates both planning and policy improvement (Jain et al., 2026).

Neither use makes every generated future equally trustworthy. Different samples from one model may represent genuine ambiguity in the partially observed scene, arbitrary generation variation, or an incorrect learned dynamics distribution. Repeated samples expose variation under that model, but a model that consistently imagines a cup sticking to an empty gripper can still be consistently wrong. This will become important when we decide which generated experiences may supervise learning.

3.1.4 What an imagined rollout actually contains

A policy-in-the-loop rollout alternates action generation and predicted observations. After the first imagined lift, the policy receives the imagined wrist and external images and selects its next chunk. The resulting distribution can drift away from real data because the policy now reacts to model errors. By contrast, replaying recorded actions through a world model tests predictions along a known action sequence. Both are useful, but they answer different questions. WEAVER's reported policy-evaluation protocol replays recorded actions; Ctrl-World studies policy-in-the-loop interaction (Guo et al., 2026b; Jain et al., 2026).

To train from a rollout, retain more than video: the conditioning context, observations, sampled actions, rewards, behavior-policy identity, world-model identity, and endpoint semantics. A terminated episode is different from an imagination horizon that ends before task completion. A terminal failure may have zero future value under the chosen task definition; a truncated rollout may need a bootstrap estimate. Confusing the two systematically changes the learning target.

This creates a familiar approximation-versus-cost question with a new consequence. A cheaper model need not preserve every pixel to be useful, but it must preserve the distinctions that determine which actions are better. For the cup, a tiny error at finger contact can reverse the

ranking of two grasp approaches even when average image error is small. Efficient sampling, latent compression, and cache reuse should therefore be assessed against downstream decisions as well as reconstruction metrics.

WEAVER reduces different parts of this cost in different ways: factorized spatial/temporal attention and patch-token dropping reduce model work, cached memory/history avoids recomputing fixed context during denoising, and rectified-flow post-training distills generated targets for fewer sampling evaluations (Jain et al., 2026). These mechanisms should not be collapsed into one claim that "memory makes the model faster." Sparse memory controls the information retained, while caching reuses a particular computation. Distillation reduces sampling work but can inherit teacher errors. A useful experiment would hold the action candidates fixed and check whether their outcome ranking survives each approximation, including difficult contact cases.

One timing example comes from WEAVER's Appendix Tables 3–4. Here NFE means number of function evaluations by the sampler, and a key–value (KV) cache stores reusable attention computations. At NFE = 16, generating a 10-second segment on one H100 takes 14.65 seconds for Ctrl-World and 4.78 seconds for WEAVER in that benchmark; disabling WEAVER's KV cache changes 4.78 to 6.54 seconds (Jain et al., 2026). The latter is about a 27% time reduction from caching. It isolates a computational benefit, not the value of memory for accuracy. Different hardware, horizons, model variants, or evaluation procedures are *not comparable*. These measurements are not a latency promise for another model and hardware configuration.

The deeper cost question is how far to trust a prediction before returning to observed evidence. Model-based policy optimization has studied short imagined branches starting from real data as a way to limit the effect of accumulated model error (Janner et al., 2019). The reason is intuitive in the cup example: a branch that starts from an observed grasp needs to predict fewer uncertain contacts than one that invents the entire approach, grasp, lift, and placement. This motivates a design tradeoff; it does not establish that a particular short horizon is valid for our task.

We now have a way to produce experience, real or imagined. Yet a sequence of observations and actions does not specify what should improve. We need rewards, returns, and a principled comparison with what the policy already expected.

3.2 From outcomes to rewards, values, and advantages

Demonstrations told the robot *which actions were taken*. Experience tells it *what happened after actions it chose*. To improve from that experience, the robot needs to turn outcomes into comparisons. We will build that comparison in three steps: define what counts as desirable, accumulate that signal over the future, and compare an attempted action with what was normally expected from the same situation.

3.2.1 The score is a modeling choice

Suppose success means that the cup rests upright on the coaster after release. A simple episodic reward is zero at intermediate steps and one at verified completion. This gives a clear outcome but a delayed signal: an early approach action receives no immediate reward even if it is necessary for success. Alternatively, dense rewards might score distance, grasp progress, or elapsed time. Such choices provide more feedback but introduce preferences that can be exploited. A robot rewarded for bringing the cup near the coaster might hover forever without releasing it.

This is why a task instruction and a reward play different roles. The instruction is part of what conditions behavior; the reward is the criterion used to assess experience. They should express compatible intentions, but one does not automatically implement the other. A trajectory can be

conditioned on "place the cup" while a poorly designed reward only measures proximity. Learning then follows the actual reward signal, including its omissions.

Let r_t be the reward for the transition from step t to step $t+1$, $\gamma \in [0,1]$ the discount per step, and T_{ep} the terminal step index. The return G_t is the discounted sum through that episode:

$$G_t = \sum_{k=0}^{T_{\text{ep}}-t-1} \gamma^k r_{t+k}. \tag{3.2}$$

We use r_t for the reward associated with the transition starting at t; other indexing conventions shift it by one. A return combines future rewards, whereas an immediate reward scores one transition. For a chunk-based decision, the reward over its executed prefix and the discount to the next decision must be aggregated consistently. One cannot use the discount per physical step as the discount per chunk without considering the chunk duration.

Worked example: Why an unrewarded approach can still be useful

Consider a three-decision task: approach, grasp-and-lift, then place-and-release. Let the rewards on a successful trial be $(0, 0, 1)$. With $\gamma = 0.9$, the returns seen from those decisions are $(0.81, 0.9, 1)$. The approach had no immediate reward but a positive future return. On a failed trial with rewards $(0, 0, 0)$, all three returns are zero. The numbers provide a way to credit earlier actions for later outcomes, although a single trajectory cannot prove which action was responsible. The temporal credit-assignment problem is precisely how to use such delayed evidence to improve decisions (OpenAI, 2018c).

Define the value function V^π as the expected return when policy π continues from the current information:

$$V^\pi(h_t, g, C) = \mathbb{E}_\pi[G_t \mid h_t, g, C]. \tag{3.3}$$

A neural critic V_ν approximates this function with parameters ν. The critic answers "how much return should I expect from here?" The policy answers "what action should I take?" Their outputs and objectives differ even if they share some representation. Policy improvement does not train the policy to become the value network (OpenAI, 2018c).

With only terminal success reward, no other reward, $\gamma = 1$, and a consistent episode definition, expected return equals success probability. Under those assumptions, a well-calibrated value of 0.6 would mean a 60% success probability. A progress score, discounted return, or hybrid progress/TD prediction does not automatically have that interpretation. Even in the binary case, a learned estimate still needs calibration before its decimal value is treated as a probability.

A value prediction compresses the answer to many possible future attempts. Starting from the same cup pose, the policy may grasp successfully in some trials and slip in others; the value is the expected return over those continuations. Change the policy, and that expectation can change even though the physical scene is identical. The superscript π in V^π reminds us which continuation behavior is being evaluated. A difficult grasp can have low value for a weak policy and high value for a more capable one.

3.2.2 How a critic learns and why a baseline helps

Let N be the number of training samples, i index a sample, and G_i^{target} denote its fixed return target. Let sg be the stop-gradient operator: it leaves a value unchanged but gives it zero derivative in this update. The critic loss $\mathcal{L}_V$ fits the targets by minimizing

$$\mathcal{L}_V(\nu) = \frac{1}{N} \sum_{i=1}^{N} \left(V_\nu(h_i, g_i, C_i) - \text{sg}(G_i^{\text{target}}) \right)^2. \tag{3.4}$$

A completed observed return supplies a Monte Carlo target. A frozen target-critic parameter copy is denoted by $\bar{\nu}$. A bootstrapped target instead uses rewards collected over a shorter interval plus another value prediction. One-step temporal-difference (TD) learning uses a target such as $r_t + \gamma V_{\bar{\nu}}(h_{t+1})$, with appropriate terminal masking and with the target treated as fixed for that update. Bootstrapping reduces the need to wait for an episode to finish, but introduces dependence on the critic's own predictions.

The reason this target is sensible is the recursive structure of return. The return is today's reward plus the discounted remainder. If the remainder is estimated by the value at the next information state, we obtain the one-step Bellman relationship

$$V^{\pi}(h_t) = \mathbb{E}_{\pi}[r_t + \gamma V^{\pi}(h_{t+1}) \mid h_t], \tag{3.5}$$

with terminal continuation value zero and task/context suppressed. The expectation includes the policy's action at h_t and the environment's response. This relationship is developed in Section 3.5 of Sutton and Barto; Section 6.1 explains the distinction between a Monte Carlo return and a bootstrapped TD target (Sutton and Barto, 2018). With a sufficient history it describes the intended expectation; a compressed or incomplete representation can introduce approximation error.

For a toy numerical update, suppose the critic predicts 0.30 before a reach, there is no immediate reward, and it predicts 0.60 after the reach. At $\gamma = 0.9$, the one-step target is 0.54. The discrepancy $0.54 - 0.30 = 0.24$ says the current prediction is lower than this bootstrapped target. It does not say we observed a successful placement, and the next-state estimate might itself be wrong. This is the distinction between a measured outcome and a learning target assembled partly from predictions.

The action-value function $Q^{\pi}(h_t, a_t)$ is expected return after choosing a_t and then following π. Define the advantage $\mathcal{A}^{\pi}$ as its difference from the state value:

$$\mathcal{A}^{\pi}(h_t, a_t) = Q^{\pi}(h_t, a_t) - V^{\pi}(h_t). \tag{3.6}$$

Task and context arguments are suppressed here, but are still present. An advantage compares an action with the policy's expected behavior in that situation. It is not simply the immediate reward, and a successful action need not have a large positive advantage if success was already nearly certain.

Worked example: An advantage is a comparison, not another reward

At one fixed cup observation, imagine only two abstract choices: approach from the left or the right. Suppose the current policy chooses each with probability 1/2, and their true expected returns are 0.8 and 0.4. Then $V^{\pi} = 0.5(0.8) + 0.5(0.4) = 0.6$, so their advantages are $+0.2$ and -0.2. The right approach can succeed sometimes and still have negative advantage because it is worse than the current average. If the policy shifts toward the left approach, both its expected return and the baseline for future comparisons change. In a real task we do not know these exact branch values; sampled outcomes and critics estimate them.

Worked example: Same success, different improvement signal

For an undiscounted success-only task, suppose the critic predicts 0.8 before an easy centered grasp and 0.3 before a difficult occluded grasp. Both trials succeed. Define the estimated advantage $\widehat{\mathcal{A}}$ as the observed return minus the critic prediction, $\widehat{\mathcal{A}} = G - V_{\nu}$. Their estimated advantages are 0.2 and 0.7. A failed difficult grasp would have advantage -0.3. This is an estimator from sampled outcomes, not proof that one individual action caused the entire outcome. Repeated data and an appropriate credit-assignment scheme are needed to distinguish good decisions from luck.

Let $d_t = 1$ at a genuine terminal transition and $d_t = 0$ otherwise. Using a fixed critic $V_{\bar{\nu}}$, define the TD residual δ_t by

$$\delta_t = r_t + \gamma(1 - d_t)V_{\bar{\nu}}(h_{t+1}) - V_{\bar{\nu}}(h_t), \tag{3.7}$$

Generalized advantage estimation (GAE) combines successive residuals with weights $(\gamma\lambda)^k$, where $\lambda \in [0, 1]$ controls how quickly later residuals are downweighted (Schulman et al., 2016). The tradeoff is between noisy long-horizon evidence and biased short-horizon predictions. At an artificial rollout cutoff, bootstrapping should reflect truncation rather than falsely declaring a terminal state. In a learned world, both the rewards and next observations may already be approximate, so a numerically stable advantage computation can still rank actions incorrectly.

3.2.3 A first use of advantages: learning from experience with RECAP

We can now return to the robot that closes its gripper too early. Autonomous trials reveal what happens under its own behavior; a person can supply a corrective reapproach; returns and value estimates provide a way to compare the recorded actions. RECAP, associated with $\pi^*_{0.6}$, combines demonstrations, autonomous experience, and human interventions using advantage-conditioned policy training (Physical Intelligence, 2025b).

The key idea is to make the improvement label an input to the policy during training. Let b_{adv} be a bin or indicator derived from the estimated advantage. Write the conditioned policy schematically as $\pi_\theta(\mathbf{a} \mid o, g, b_{\text{adv}})$. Here o is the current observation; the available history and any other fixed inputs are suppressed. The policy sees which recorded behavior was associated with which condition. At inference, requesting the improvement condition favors the corresponding learned behavior. This resembles using a task label to select a behavior family, except the label now describes estimated quality relative to a baseline. It is still necessary to train the conditional mapping; merely adding the desired label to an arbitrary policy would not create the skill.

RECAP uses a binary improvement indicator and treats expert corrections as positive examples under its procedure (Physical Intelligence, 2025b). Its update has a supervised form, but the labels incorporate reward-bearing experience, so it is more than cloning an undifferentiated set of demonstrations. At the same time, it does not use the policy-gradient surrogate developed in the next two sections. We now have one concrete route from advantages to improved behavior. The next route asks how to directly adjust a stochastic policy in a direction that increases expected return.

3.2.4 Likelihood is not reward

The policy also assigns a likelihood to actions, but this is a different quantity. In continuous action space $\pi_\theta(a \mid h)$ is a *density*; the probability of a region is its integral over that region. The probability of exactly one real-valued action is usually zero, and a density may exceed one. A log density can be positive or negative and depends on coordinates and units. None of this says whether the action is good.

An early-close grasp can have high likelihood because it is common under the current policy and still receive a poor return. A rare successful recovery can have low likelihood and high advantage. Reinforcement learning uses reward-bearing evidence to change the distribution; it does not declare actions good because the model already prefers them. This distinction is particularly useful when translating from language-model training, where log probabilities and reward scores often appear in the same loss expression.

3.3 Why a policy gradient can learn without differentiating physics

3.3.1 The direction we want before the derivative we need

Continue the two-approach example. Let p be the probability of choosing left and $J(p)$ the corresponding expected return. Then

$$J(p) = 0.8p + 0.4(1 - p) = 0.4 + 0.4p. \tag{3.8}$$

Increasing p improves the expected return. There is no need to change the physical outcome of a particular left attempt by a differentiable amount; we change how frequently that kind of attempt is selected. The real difficulty is that a neural policy has many parameters and the two branch returns are not known in advance. We need a gradient estimator built from sampled actions and their outcomes (OpenAI, 2018a).

This is why the policy's log likelihood appears in reinforcement learning. Its gradient tells us how to change the weights to assign more density to an observed action. The return or advantage tells us whether and how strongly to use that direction. The log likelihood provides a direction in parameter space; it does not supply the task's notion of goodness.

3.3.2 A gentle log-derivative argument

For a fixed observation, imagine sampling an action from π_θ and receiving a score $f(a)$ that we can evaluate but cannot differentiate. Assume that f is independent of θ, differentiation can pass under the integral, and the policy has suitable parameter-independent support. With ∇_θ denoting differentiation with respect to the policy parameters,

$$\begin{aligned} \nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[f(a)] &= \int f(a)\nabla_\theta \pi_\theta(a)\, da \\ &= \mathbb{E}_{a\sim\pi_\theta}[f(a)\nabla_\theta \log \pi_\theta(a)]. \end{aligned} \tag{3.9}$$

The second line uses $\nabla\pi = \pi\nabla \log \pi$. We differentiate the probability assigned to a sampled action, not the environment's response to that action. In a trajectory, the same idea applied to the product of policy probabilities yields a sum of score terms. The environment's transition factors do not contribute parameter derivatives when the environment is fixed and independent of θ. Reward-to-go and baselines then lead to the usual policy-gradient estimator (OpenAI, 2018a).

Subtracting a baseline depending on the observation but not the sampled action leaves the expected score-function gradient unchanged, since $\mathbb{E}_a[\nabla_\theta \log \pi_\theta(a \mid h)] = 0$. A good value baseline can reduce variance. This is why a critic is useful even when the policy does not differentiate through the critic or environment during its update.

The baseline has a practical interpretation in the cup example. Easy scenes tend to produce successes under many actions, whereas difficult scenes produce failures under many actions. Comparing each outcome against its scene-dependent expectation helps separate "this was an easy starting point" from "this action did better than expected here." It does not remove all randomness or identify causality from one trial; it improves the statistical signal used across trials.

Let θ_{old} denote the parameters of the policy that collected a batch of N samples. Hold each history h_i, sampled action a_i, and estimated advantage $\widehat{\mathcal{A}}_i$ fixed. A local gradient surrogate S is

$$S(\theta) = \frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i \mid h_i)\, \widehat{\mathcal{A}}_i. \tag{3.10}$$

At the collection policy, its gradient estimates a policy-gradient direction under the relevant assumptions. It is not the full expected-return objective for arbitrary θ, and repeatedly optimizing it on stale data does not maintain that interpretation. Compact expressions involving log policy and advantages should be read with these sampling and stop-gradient conventions in mind. Differentiating the sampling distribution and the advantage again inside the same empirical surrogate would be a different computation.

3.3.3 Old and current policies in PPO

One gradient step changes the policy that generated the data. After several steps, the batch still contains the same actions, but the current policy would choose them with different frequencies. Reusing the batch can save collection effort, yet it needs a controlled comparison with the collection policy. This motivates recording an old-policy snapshot and asking, for each sampled action, how its density has changed. Proximal Policy Optimization (PPO) uses that comparison in a local surrogate and limits the incentive to make large changes (OpenAI, 2018b). Define $\rho_i(\theta)$ as the current-to-behavior action-density ratio:

$$\rho_i(\theta) = \frac{\pi_\theta(a_i \mid h_i)}{\pi_{\theta_{\text{old}}}(a_i \mid h_i)}. \tag{3.11}$$

Many papers call this $r(\theta)$. We use ρ to avoid confusing it with the environmental reward r_t. The denominator belongs to the fixed policy that collected the batch; the numerator changes during training. Their densities must use the same action coordinates and conditioning. At identical parameters, the ratio is one.

For intuition, temporarily return to discrete choices. If a left approach had probability 0.5 when the data were collected and probability 0.6 now, its ratio is 1.2. It receives 20% more weight in that action comparison. In continuous space the analogous comparison uses densities. This action ratio does not by itself correct every change in the distribution of visited states, which is one reason PPO is treated as a local update with fresh collection rather than a license to optimize one batch indefinitely.

Let $\epsilon_{\text{clip}} > 0$ be the clipping width, and let $\text{clip}(x, l, u)$ restrict x to $[l, u]$. PPO maximizes the clipped surrogate S_{PPO} (Schulman et al., 2017)

$$S_{\text{PPO}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \min\left[\rho_i \widehat{\mathcal{A}}_i, \text{clip}(\rho_i, 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \widehat{\mathcal{A}}_i\right], \tag{3.12}$$

For positive advantage, increasing the ratio helps until the clipped branch removes the incentive to increase it further. For negative advantage, decreasing the ratio helps until the lower clip removes that incentive. The minimum is sign-sensitive; it does not simply clamp every gradient whenever a ratio leaves the interval.

Worked example: What clipping does and does not prevent

Let $\epsilon_{\text{clip}} = 0.2$ and $\widehat{\mathcal{A}} = 0.7$. Raising a ratio from 1.0 to 1.3 would raise the unclipped contribution from 0.7 to 0.91, but the clipped objective uses 0.84. There is no further reward from that term for going above 1.2. For $\widehat{\mathcal{A}} = -0.3$ and ratio 0.7, the minimum selects -0.24 rather than -0.21, so further reducing that ratio earns no benefit from the term. These plateaus do not project the policy into a hard distance ball. Other samples, shared parameters, or other loss terms can still move its distribution, and there is no universal policy-distance guarantee from clipping alone.

We now have the ingredients for a policy update: collected actions, estimated advantages, and an old/current density comparison with clipping. A simple Gaussian policy can evaluate that density

directly from its mean and covariance. A flow policy instead defines its action distribution through a sequence of transformations. Generating a sample follows those transformations; evaluating exact density must also account for how they expand or compress probability mass. Consequently, easy sampling does not imply cheap exact likelihood evaluation (Lipman et al., 2023; McAllister et al., 2025a).

This is where the earlier flow-matching loss becomes useful again. We know how to evaluate a velocity-prediction error on a chosen action without reconstructing its original sampling path. FPO uses differences of such losses to build a surrogate for the old/current comparison. The next section will first follow that computational recipe and then explain the more delicate probabilistic interpretation.

3.4 Flow Policy Optimization: using a loss difference as a surrogate

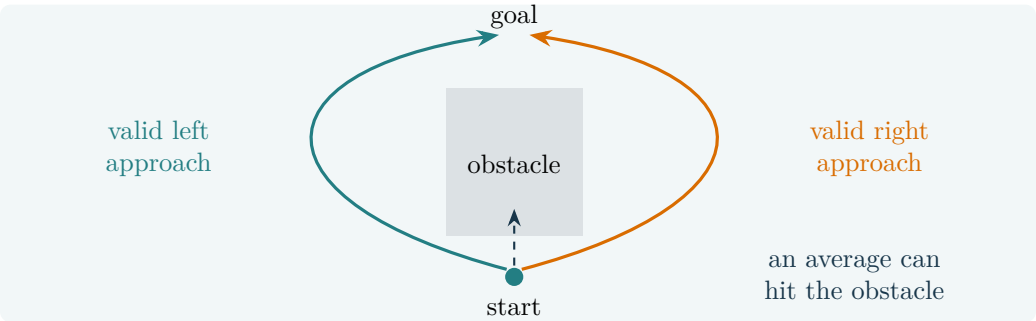


Figure 3.5: Keep both ways around the obstacle. Separated valid action modes need not have a valid average. A flexible distribution can retain both; rewards must then favor useful samples (McAllister et al., 2025a).

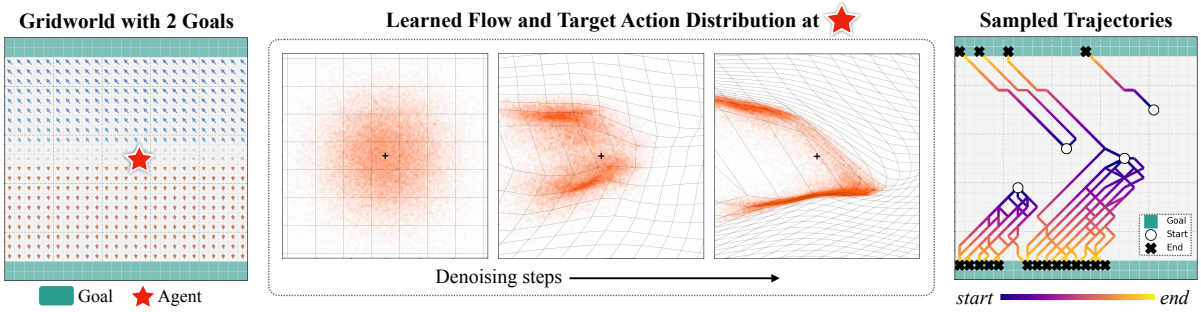


Figure 3.6: Multiple useful action modes. Figure 1 from McAllister et al. (2025a). CC BY 4.0.

Reading the paper figure. Find the starred state and compare the evolving sample distribution with the possible trajectories. The authors' GridWorld example shows how optimization reshapes a multimodal action distribution.

3.4.1 Score a collected action with the flow model

Flow Policy Optimization (FPO), introduced in *Flow Matching Policy Gradients*, adapts a PPO-style update to flow policies using conditional flow-matching losses (McAllister et al., 2025a). The action used as a target is now a sampled rollout action or chunk. It need not be an expert demonstration. The external experience supplies an advantage that tells the update whether to favor or disfavor that sample.

For one collected pair $(c_i, \mathbf{a}_i)$, draw M noise/time pairs $(\epsilon_{im}, \tau_{im})$, indexed by m. Form the same noised action and reference velocity for the old and current networks. With ℓ_θ denoting the per-sample flow loss under the chosen weighting convention, define the Monte Carlo loss estimates $\widehat{\mathcal{L}}_{\theta,i}$ and $\widehat{\mathcal{L}}_{\text{old},i}$, and the FPO surrogate ratio $\tilde{\rho}_i$:

$$\begin{aligned}
\widehat{\mathcal{L}}_{\theta,i} &= \frac{1}{M}\sum_{m=1}^{M} \ell_\theta(c_i, \mathbf{a}_i; \epsilon_{im}, \tau_{im}), \\
\widehat{\mathcal{L}}_{\text{old},i} &= \frac{1}{M}\sum_{m=1}^{M} \ell_{\theta_{\text{old}}}(c_i, \mathbf{a}_i; \epsilon_{im}, \tau_{im}), \\
\tilde{\rho}_i(\theta) &= \exp\left(\widehat{\mathcal{L}}_{\text{old},i} - \widehat{\mathcal{L}}_{\theta,i}\right).
\end{aligned} \tag{3.13}$$

The paired comparison is important. If the two networks are identical, the same inputs give the same losses and the estimated ratio is exactly one up to numerical error. Using different noise draws for the two sides would add a spurious difference even before any policy change. The old weights are frozen; if the noise/time samples are refreshed, the old loss must be evaluated consistently on those refreshed samples.

Substitute $\tilde{\rho}$ into the clipped objective in Equation (3.12). The surrogate ratio itself has derivative

$$\nabla_\theta \tilde{\rho} = -\tilde{\rho}\nabla_\theta \widehat{\mathcal{L}}_\theta. \tag{3.14}$$

When the clipped objective is not on a plateau, positive advantage means that maximizing the surrogate tends to reduce the current loss on that action. For negative advantage, the local direction is reversed. The clipping and interactions among samples qualify this intuition; the algorithm does not independently set the likelihood of every action.

Worked example: A paired loss calculation

Suppose a collected cup-lift chunk has estimated advantage 0.7. On shared noise/time samples, let the frozen policy loss be 1.10 and the current loss be 0.95. The surrogate ratio is $e^{0.15} \approx 1.162$. With clipping width 0.2, this term remains below the positive-advantage plateau. If the current loss drops to 0.70, the ratio becomes $e^{0.40} \approx 1.492$ and the positive contribution is clipped at $1.2 \times 0.7 = 0.84$. In particular, 1.162 is not an estimated task-success probability.

3.4.2 Why the ratio is a surrogate

A flow loss can be related to a variational lower bound on log likelihood, but the relationship depends on the noise schedule, prediction parameterization and loss weighting. The gap between a bound and the corresponding likelihood can change during training. Thus an exponentiated loss difference is not an exact likelihood ratio. Finite noise sampling adds estimation error, and changing the loss normalization changes the exponent. Appendix A develops the argument and defines the evidence lower bound (ELBO). The operational rule is simple: retain the word *surrogate*, record the loss convention, and verify improvement outside the training objective.

3.4.3 Where the gradients go

The computational graph for an update contains the current velocity network evaluated on noised versions of collected actions. Observations, actions, advantages, old losses, and the reference velocity are treated as fixed data. Gradients pass through $\widehat{\mathcal{L}}_\theta$ to θ. They do not need to pass through robot transitions, the world model that generated the rollout, or the ODE sampling

trajectory that originally produced the action. This is the score-surrogate route to optimization, not differentiable control through a simulator (McAllister et al., 2025a,b).

In an automatic differentiation system, a differentiable world model need not be part of the policy-loss gradient calculation. A proposed implementation should explicitly stop gradients through the stored rollout and advantage estimates. A critic can be trained in a separate step using its own targets. The policy and critic may improve together operationally while having different parameter updates.

FPO's original control experiments train policies from scratch; they do not validate a complete pretrained, in-context, learned-world-model pipeline (McAllister et al., 2025a). Adopting its update for that pipeline is a design proposal. The proposal must be tested against simpler alternatives before the added complexity is justified.

3.4.4 Online is not synonymous with on-policy

Online versus offline describes whether new experience is collected during learning or a fixed dataset is used. *On-policy versus off-policy* describes the relationship between the behavior that generated data and the policy update. An on-policy method collects recent behavior-policy batches and limits how far updates move before recollection. An off-policy method is designed to use experience from other or older behavior policies. Both kinds update network weights.

A fixed real dataset can train a world model. A policy can then collect fresh imagined trajectories inside that fixed model and perform an on-policy FPO-style update. Collection is online relative to the learned environment while there is no new physical interaction. Such data are fresh for the policy in that model; they are not automatically representative of the real robot. Conversely, online physical collection can feed an off-policy replay algorithm. The two labels answer different questions and should both appear in a training specification.

Checkpoint — One complete update. Explain why storing an old loss without its noise/time convention is insufficient; why $\widehat{\mathcal{A}}$ is fixed during the policy step; and why the exponentiated loss difference is a surrogate rather than an exact likelihood ratio. Then identify which objects require new values when the behavior-policy snapshot changes.

3.5 Two published ways to learn from imagined experience

FPO supplies an update mechanism, but it does not decide how the learned environment is trained, how reward is checked, or when model refreshes occur. RISE and VLAW make these surrounding choices concrete. Neither uses FPO as its published policy update. They should be read as distinct system designs whose ideas may inform a later integration.

3.5.1 RISE: a fixed learned environment during policy improvement

RISE combines a dynamics model with a progress/value model. It warms up the policy on real experience, then produces imagined data labeled with advantages for advantage-conditioned flow training. Its dynamics and value models remain frozen during the self-improvement loop (Yang et al., 2026). This fixes the learned environment while the policy changes, making the loop easier to analyze but leaving model errors available for exploitation.

See Yang et al. (2026), Figure 4.

Let $\mathcal{V}_{\text{RISE}}$ denote RISE's task-conditioned progress/value predictor. Its chunk score $\widehat{\mathcal{A}}_{\text{RISE}}$ averages predicted score changes from the initial observation across H future observations. In our

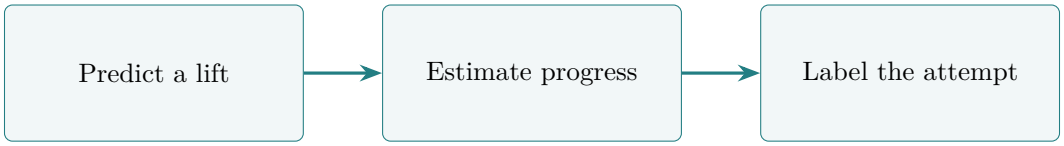


Figure 3.7: Prediction and evaluation can fail separately. A progress estimate describes the predicted event; it does not independently prove that event was physically possible (Yang et al., 2026).

notation,

$$\widehat{\mathcal{A}}_{\mathrm{RISE}}(o_t, \mathbf{a}_t) = \frac{1}{H}\sum_{k=1}^{H}[\mathcal{V}_{\mathrm{RISE}}(\widehat{o}_{t+k}, g) - \mathcal{V}_{\mathrm{RISE}}(o_t, g)]. \tag{3.15}$$

It discretizes advantages into conditioning bins and requests the preferred bin when sampling. The progress/value model combines temporal-progress supervision with TD learning. This operational chunk advantage should not be silently equated with every textbook advantage estimator, and its score is not automatically a calibrated success probability. The reported implementation limits consecutive imagined interactions to two to constrain accumulated error (Yang et al., 2026).

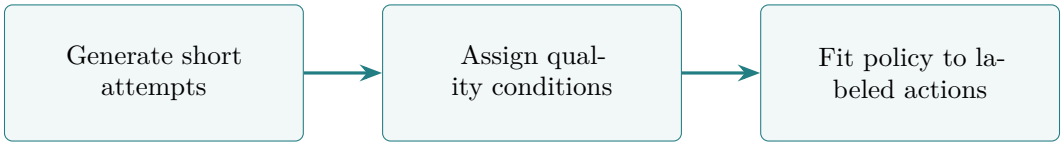


Figure 3.8: Improve behavior in a fixed learned environment. The quality condition enters supervised action learning. This is distinct from the flow-loss ratio used by FPO (Yang et al., 2026).

See Yang et al. (2026), Figure 5.

For the cup, short imagination may ask whether an approach and close sequence improves the predicted situation. Keeping rollouts short reduces how much generated history feeds back into itself, but it may cut off the eventual slip or release failure. A value model can estimate what follows the cutoff, yet that simply moves part of the approximation burden into the value estimate. Short horizons and bootstrapping are complementary design choices, not independent guarantees of reliability.

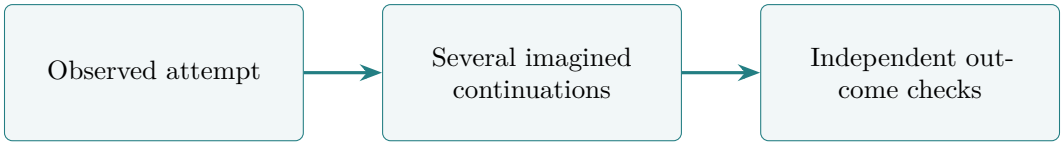


Figure 3.9: Branch from observed experience. More generated samples are useful only if their consequences and labels are reliable. Quantity does not turn prediction into observation (Guo et al., 2026a).

See Guo et al. (2026a), Figure 1.

3.5.2 VLAW: refresh the simulator with real successes and failures

VLAW alternates real rollout collection, world-model adaptation, synthetic generation, filtering, and policy training. It refreshes Ctrl-World using accumulated real successes and failures mixed with DROID. The reported reward model is fine-tuned on first-iteration real success labels. Policy training uses successful real trajectories plus synthetic trajectories accepted by the reward model (Guo et al., 2026a). This is success-filtered flow-matching training, not FPO or an obligatory separate TD-critic update.

See Guo et al. (2026a), Figure 3.

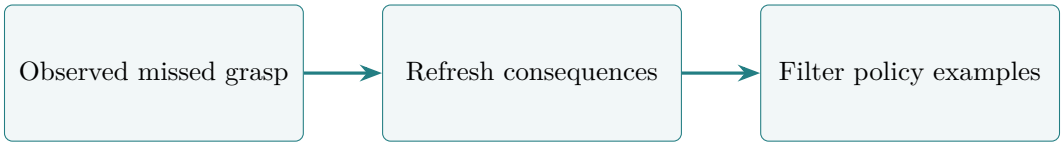


Figure 3.10: A failure can teach the dynamics. A failed grasp is useful evidence for the world model even when it is excluded from positive-only policy imitation (Guo et al., 2026a).

Real failures are particularly valuable for a world model trained mainly on successful demonstrations. An early-close cup attempt teaches that the cup may remain on the table while the gripper rises. This negative outcome is a valid dynamics target even though it is not a successful policy demonstration. The same episode can therefore belong in the dynamics dataset and be excluded from positive-only policy training. Data utility depends on which model is being trained.

Table 3.1: Published update mechanisms remain distinct. This table summarizes algorithmic roles, not a performance ranking.

Method	Experience-to-policy mechanism	Learned environment or feedback schedule
RECAP	Advantage-conditioned policy training	Values and real experience/corrections support improvement labels.
RISE	Advantage-conditioned flow training on imagined data	Dynamics and value frozen during the policy-improvement loop.
VLAW	Flow matching on successful real and accepted synthetic data	World model refreshed with accumulated real data; reward training uses first-iteration labels in reported experiments.
FPO	Advantage-weighted clipped surrogate from old/current flow losses	Update rule does not itself specify or validate a learned world model.

These systems suggest several possible integration choices. One could begin with success filtering, add advantage conditioning, or test FPO as a separate optimizer. Combining all of them by name would obscure the actual objective. The first implementation should state precisely which data enter each loss, which parameters are fixed, and which evidence determines whether the candidate is adopted. The decisive remaining issue is whether the imagined successes deserve trust.

3.6 The false-success trap and the evidence that breaks it

3.6.1 An impossible grasp can look successful

Return to the early-close failure. Suppose the generated video depicts the cup attaching to the gripper even though the fingers miss it. The cup then appears upright on the coaster. A task evaluator may correctly recognize that the *depicted final scene* satisfies the task while the video is wrong about the consequences of the actions. An optimistic reward model may accept the sample and training may reinforce the associated behavior.

Two questions must therefore remain distinct. Does the observed or depicted outcome satisfy the task? And is the predicted future consistent with what those actions could cause from that initial state? A visual outcome label answers the first question more directly than the second. The second requires action-aligned evidence and may remain uncertain even after careful video inspection. A generated trajectory is not a measured transition.

There are several possible errors. The world model may hallucinate contact. The evaluator may miss a visible failure. The action adapter may encode the wrong frame. The task definition may

reward lifting while the actual goal requires stable placement. These diagnoses lead to different fixes. A single scalar penalty cannot identify which component failed.

3.6.2 The VLAW reward audit: keep the denominator

VLAW Appendix C, Table 3 reports an audit of 40 human-labeled trajectories. Direct yes/no decisions are compared with requiring the model's "yes" token probability to exceed 0.8 (Guo et al., 2026a). Table 3.2 restates the counts and shows the arithmetic, without turning the audit into a physical-validity certificate.

Table 3.2: VLAW reward audit (40 trajectories): source counts from Appendix C, Table 3, v2 (Guo et al., 2026a); rates below are calculated from those counts. Positive means accepted as successful.

Quantity	Direct yes/no	Threshold > 0.8
True positives (TP)	15	10
False positives (FP)	8	2
False negatives (FN)	7	12
True negatives (TN)	10	16
Accepted trajectories	23	12
False discoveries: FP/(TP+FP)	$8/23 = 34.8\%$	$2/12 = 16.7\%$
False-positive rate: FP/(FP+TN)	$8/18 = 44.4\%$	$2/18 = 11.1\%$
Success recall: TP/(TP+FN)	$15/22 = 68.2\%$	$10/22 = 45.5\%$
Accuracy: (TP+TN)/40	$25/40 = 62.5\%$	$26/40 = 65.0\%$

For positive-only policy training, the accepted subset is especially relevant. The stricter filter accepts 12 trajectories, two of which are human-labeled failures: 2/12, or approximately 16.7% false discoveries in that sample. This is different from asking what fraction of the 18 actual failures slipped through, which gives 2/18. Accuracy combines accepted successes and rejected failures over all 40 examples and answers a third question.

The reduction in false acceptance comes with a tradeoff: 12 of 22 human-labeled successes are rejected. A conservative filter can improve the purity of its accepted data while reducing coverage of useful successful behavior. Moreover, the accepted sample is small; its observed fraction is not a population guarantee. If the policy or world model changes the distribution of generated videos, the error rates may change too.

A token probability above 0.8 is a property of the evaluator's output distribution. It is not an 80% guarantee of task success and certainly not an 80% guarantee of action-conditioned physical correctness. Calibration would need a defined target event and independent labels on a relevant distribution. Human video labels are useful for visible task outcomes, but reviewers can miss subtle physics errors. Agreement between two models is also limited when they share training data, representations, or blind spots.

3.6.3 Correct the component for which evidence exists

If the cup did not move in an independently observed execution, align the executed actions and measured future observations and use them to correct the world model. The target says what actually happened. A judgment that the generated video is "unacceptable" does not by itself specify the correct future image sequence. For the evaluator, supply independently checked task labels and audit the resulting errors on held-out examples. These are distinct supervision channels.

An unverified synthetic success should be withheld from positive policy training. That does not automatically justify a negative policy update. The proposed action might have been reasonable while the simulator was wrong, or it might truly have failed. Without trustworthy outcome evidence, its advantage is unresolved. Treating every simulator inconsistency as proof of a bad action can discard useful behavior and train against artifacts of the checker.

For our proposed system, useful safeguards could include action/time alignment checks, justified geometric or contact constraints, observed-versus-generated provenance, and explicit uncertain labels. These are design proposals beyond the published VLAW success filter. They should be evaluated for both false acceptance and false rejection, since an overly restrictive validity checker can suppress successful strategies.

Worked example: An evidence-directed correction

Suppose three records concern the same cup attempt: a command log, a generated video showing a lift, and an external camera recording showing a miss. The recording, if correctly synchronized, contradicts the prediction. It supplies a dynamics target for the actual command. If the evaluator accepts the real miss video, that additionally exposes an evaluator error. If it accepts only the generated lift video, it may be judging depicted success correctly while remaining unable to verify physical validity. The policy update should use the trusted real outcome under the chosen reward rule, not an arbitrary penalty for the existence of disagreement.

Without a robot, held-out recordings permit retrospective checks of known actions and observed futures. They cannot establish the outcome of every newly proposed action. Device-free work can nevertheless establish valuable prerequisites: interface consistency, correct estimator arithmetic, provenance completeness, calibration on available labels, and clearly specified failure tests. These are evidence of preparation and bounded behavior, not evidence of physical policy improvement.

3.7 Putting the learning loop together

The learning loop can now be stated with explicit interfaces. Demonstration context helps specify a new task or behavior. A flow policy proposes action chunks. An action-conditioned world model predicts consequences. Reward and value estimates turn those consequences into training signals. A chosen policy update changes persistent weights, and independent evaluation determines whether that candidate represents progress. The phrase *In-Context World Model* does not by itself specify whether demonstrations condition the policy, the dynamics, or both; its meaning must be specified for each system.

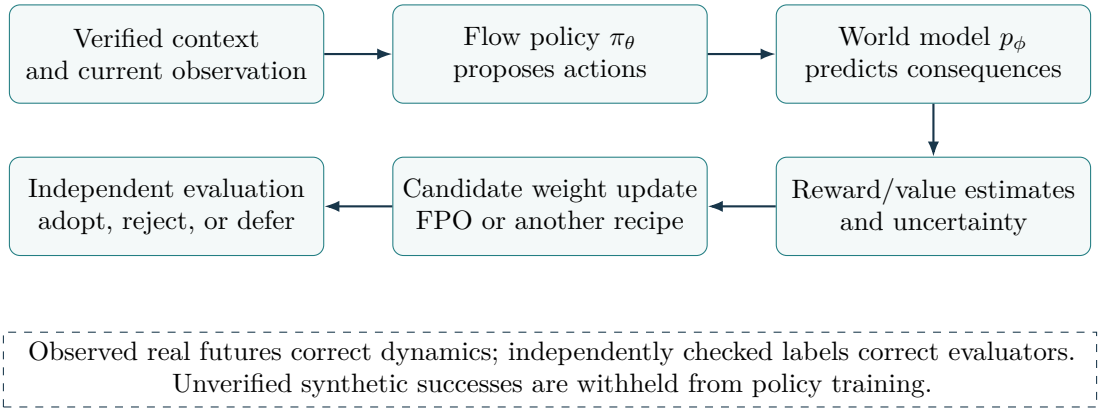


Figure 3.11: A possible learning loop combining the components developed in this chapter. Arrows show information flow; the policy update uses stored experience rather than gradients through the world model.

Three boundaries organize the proposal. Inference-time adaptation changes context features while the deployed weights remain fixed. Persistent learning changes a candidate checkpoint using retained data and a specified objective. Acceptance of that checkpoint requires evidence outside the simulator and scorer that supplied its training feedback. A single demonstration, a higher imagined return, and an independently measured improvement are different achievements.

To check understanding, trace one cup episode from beginning to end. Identify the observation, the action convention, the demonstration information, the sampled chunk, its executed prefix, the reward rule, the value target, the advantage estimator, the policy-loss gradient, and the evidence supporting the outcome. If one of those nouns has no concrete record or definition, the integration is not yet specified. Chapter 4 examines the prediction model and its data contract. Chapter 5 develops the training and evaluation cycle. Chapter 6 turns them into feasible first research projects.

Checkpoint — Explain the loop in five minutes. Use the cup example without paper names. Explain where inputs change, where weights change, which predictions can be wrong, and which observations could disprove an apparent improvement. Then attach each paper name only to the component or learning recipe it actually contributes.

3.8 Exercises

Exercise 3.1. For rewards 0, 0, 1 and discount 0.9, compute the return at the first step. If the value estimate there is 0.6, compute return minus value. What changes if the success was only imagined?

Exercise 3.2. Old and current estimated flow losses are 0.8 and 0.6. Compute the FPO surrogate ratio. Why is this not proof that the action probability increased by that exact factor?

Chapter 4

From Video Prediction to a Controllable Manipulation Model

The cup example in Chapter 3 established a complete learning loop. Now make the task less forgiving. A gripper holds a connector just above its socket. The camera suggests that the parts are aligned, but the connector could be tilted, the cable could be pulling sideways, or one edge could already be touching the fixture. The next command is only a small movement. Its consequence could be insertion, harmless contact, or a jam.

The question is no longer simply whether we can generate the next image. It is whether a compact model preserves enough information to distinguish actions that matter. We will answer it in stages: give controls a precise meaning, identify the information available before execution, choose a representation, and expose contact errors.

4.1 First decide what question the model answers

A predictive world model answers a conditional question: given what we have observed and a specified action, what might happen? A policy answers a different question: given what we have observed and want to achieve, what action should we choose? A system may share features between these functions, but their interfaces remain different.

Use the same variables as Chapter 3: o_t is the current observation, h_t the available history, g the task specification, and C optional demonstration context. The policy parameters are θ; the world-model parameters are ϕ. For a one-action decision,

$$a_t \sim \pi_\theta(\cdot \mid h_t, g, C), \tag{4.1}$$

$$\widehat{o}_{t+1} \sim p_\phi(\cdot \mid h_t, a_t). \tag{4.2}$$

An observed future is o_{t+1}; a generated future is $\widehat{o}_{t+1}$. Task information belongs in the policy because insertion and withdrawal require different choices. Changing the goal alone should not change the physical consequence of the same action from the same state.

There are legitimate reasons to condition dynamics on additional context: an earlier interaction might reveal stiffness, friction, or an unseen fixture configuration. Such information changes our estimate of the world. The distinction is between information about physical conditions and a desired outcome. If a generator uses "success" as an input and paints success regardless of the action, it has answered the wrong question for policy evaluation.

A policy that jointly predicts actions and future observations may be useful for choosing behavior, but it is not automatically a simulator accepting arbitrary externally chosen controls. A demonstration-conditioned policy need not predict futures at all. Instant Policy and SynthICL illustrate learning an action policy whose behavior changes with demonstration context (Qian et al., 2026a; Vosylius and Johns, 2025). Here their role is to explain task specification; a dynamics model has a separate predictive responsibility.

Checkpoint — Three arrows. For a new diagram, identify the arrow that chooses the action, the arrow that predicts its consequence, and the arrow that defines success. Shared tokens or shared weights do not erase these three responsibilities.

4.2 An action is a physical contract

"Move down by two millimetres" is incomplete. Down relative to the camera, the robot base, or the connector? Is the number a displacement, a velocity, or a target position? How long is it applied, and which controller turns it into motor commands? These details determine the transition that the world model must learn.

Worked example: The same numbers, two different motions

Suppose the command is a tool-frame displacement $\Delta\mathbf{p}^{\mathrm{T}} = (0, 0, 0.002)$ metres. Let $\mathbf{q}_t$ be the robot configuration and $\mathbf{R}_{\mathrm{BT}}(\mathbf{q}_t)$ the rotation from tool coordinates to base coordinates. The corresponding base-frame displacement, denoted by $\Delta\mathbf{p}^{\mathrm{B}}$, is

$$\Delta\mathbf{p}^{\mathrm{B}} = \mathbf{R}_{\mathrm{BT}}(\mathbf{q}_t)\Delta\mathbf{p}^{\mathrm{T}}. \tag{4.3}$$

A tilted tool can move diagonally in base coordinates even though only the third entry of the local displacement is nonzero. By contrast, a base-frame command $(0, 0, 0.002)$ moves along the base's third coordinate axis. The translation of the tool origin is absent from Equation (4.3) because it transforms a displacement vector; transforming a point would also require translation.

If 0.002 instead means metres per second, applying it for 0.1 seconds produces a nominal displacement of 0.0002 metres. The requested displacement differs by a factor of ten. A model cannot recover the intended interpretation from a normalized number alone. Normalization constants must therefore travel with the action schema.

The applied motion also depends on the controller. A compliant controller may yield at contact while another controller tracks the same target more rigidly. Let κ collect controller and environment settings, Δt be the physical interval, and $F_{\Delta t}$ be a physical transition map over that interval. Let $\mathcal{O}$ map physical state to an observation. Process and observation errors are denoted by ε_t^s and ε_t^o:

$$s_{t+1} = F_{\Delta t}(s_t, a_t; \kappa) + \varepsilon_t^s, \qquad o_t = \mathcal{O}(s_t) + \varepsilon_t^o. \tag{4.4}$$

The additive errors are a teaching abstraction in a chosen coordinate representation. A probabilistic predictor represents uncertainty over relevant variables it cannot observe; a point predictor may average over them. Mixing incompatible controllers without recording their settings can give apparently identical training inputs contradictory targets.

4.2.1 Put observations on the correct side of the action

The integer t indexes control steps. Let $t_{\mathrm{phys},t}$ be the physical timestamp of step t. The observation o_t precedes action a_t, which is applied over $[t_{\mathrm{phys},t}, t_{\mathrm{phys},t} + \Delta t)$. The next observation is o_{t+1}. Real cameras have exposure times and transport delay; a timestamp on receipt is not necessarily the measurement time. The predictor should be trained with the action that actually governed the interval, including any controller clipping or interruption that is available in the log.



Prediction is issued before the rightmost observation exists.
A later frame may supervise training; it cannot be a deployment input.

Figure 4.1: Observation and action timing. Physical time Δt is separate from the noise or integration coordinate inside a generative sampler.

A policy might propose ten controls but execute only two before observing again. The world model must receive the executed prefix when explaining the recorded next observation. When imagining the unexecuted suffix, it is making a counterfactual prediction. The sampler's number of neural evaluations changes computational cost; the executed action duration changes physical exposure. They should never share an ambiguous "step" field.

4.3 Learn from future frames without needing them at deployment

Video provides abundant evidence of change, but a change is not the same as a command. Seeing a connector move into a socket tells us both about the gripper motion and about the object's response. An encoder trained to compress the entire transition can hide both in its latent action. Such a code can reconstruct a future extremely well while being hard to predict from a robot command.

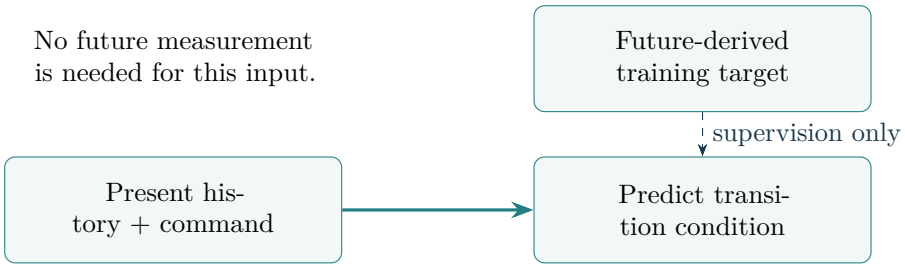


Figure 4.2: Keep tomorrow out of today's inputs. A training encoder may inspect the future. The deployed action adapter must use information available before execution (Lv et al., 2026).

SkeLAM separates skeleton-grounded kinematic information from residual visual transition information. Skeleton supervision and swap reconstruction encourage compatible motion representations; branch dropout reduces dependence on future-derived conditions. A robot-action adapter supplies the kinematic condition, with a null residual at inference (Lv et al., 2026, Sec. 3 and App. B). Its Appendix C.3 evaluates recorded-action open-loop replay: 105 episodes, only nine real successes. This provides evidence about prediction and ranking on that dataset; it does not test closed-loop policy improvement.

To isolate the general issue, let E_{tr} be a fixed transition encoder and $z^\star = E_{\text{tr}}(o_t, o_{t+1})$ its latent training target. Let f_ω be an action adapter with parameters ω, producing $\widetilde{z} = f_\omega(h_t, a_t)$ before the future is known. On an aligned dataset $\mathcal{D}$, a simple adapter loss is

$$\mathcal{L}_{\text{adapt}}(\omega) = \mathbb{E}_{\mathcal{D}}\left[\left\|f_\omega(h_t, a_t) - \text{sg}(z^\star)\right\|^2\right]. \tag{4.5}$$

This teaching loss updates the adapter, not the fixed target encoder. If the target includes an unpredictable outcome, the adapter cannot infer it reliably. Squared-error regression may instead predict an average that describes no useful physical transition.

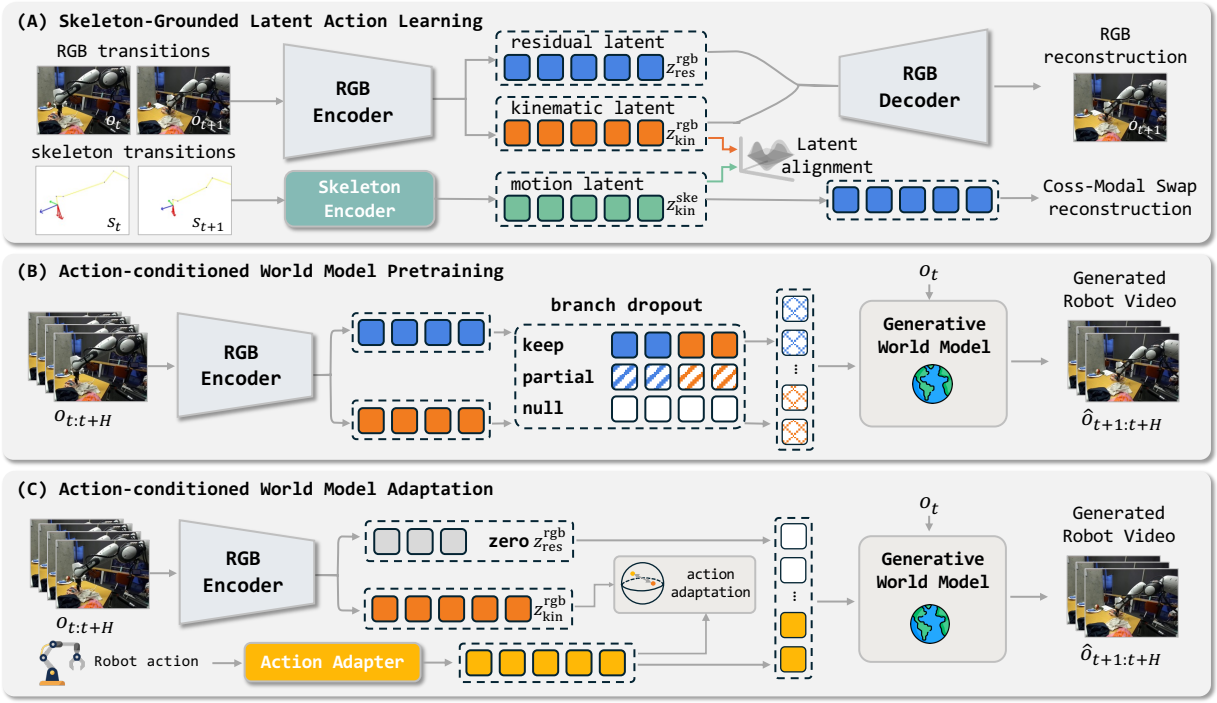


Figure 4.3: Training information and deployment inputs. Figure 1 from Lv et al. (2026). CC BY 4.0.
Reading the paper figure. Compare the transition-encoding branch with the robot-action adapter. Future observations can supervise representation learning, but the deployed adapter must produce its condition before the future is observed.

For insertion, imagine two identical commands from visually similar starts: one seats the connector and one catches an edge. If the latent target includes the fact of seating, the command alone cannot determine it without more state information. We can improve observability, represent several plausible futures, or restrict the latent to information the command actually specifies. We cannot solve missing information simply by increasing reconstruction accuracy.

Checkpoint — Availability audit. For every conditioning tensor, write when it becomes available. Future frames, success labels, simulator-only states, and human corrections may be valid training targets while remaining unavailable deployment inputs.

4.4 Compress what the next decision needs

A raw image spends many pixels on texture and lighting. A compact representation can reduce prediction cost, but the connector's tiny alignment error may occupy fewer pixels than an irrelevant shadow. Compression is therefore a modeling choice about retained information, not merely a file-size reduction.

Let E_α be an observation encoder with parameters α, D_β a decoder with parameters β, and $z_t = E_\alpha(o_t)$ an observation latent. Let P_ϕ be a point predictor of the next latent. It uses the most recent L latents, denoted by $z_{t-L+1:t}$, and the applied action. Define d_{image} and d_{latent} as chosen nonnegative reconstruction and prediction losses. A generic two-stage objective is

$$\min_{\alpha,\beta} \quad \mathbb{E}\, d_{\text{image}}\big(D_\beta(E_\alpha(o_t)), o_t\big), \tag{4.6}$$

$$\min_{\phi} \quad \mathbb{E}\, d_{\text{latent}}\big(P_\phi(z_{t-L+1:t}, a_t), \operatorname{sg}(E_\alpha(o_{t+1}))\big). \tag{4.7}$$

The expectations are over training observations and aligned transitions. The encoder and decoder are fixed in the second stage. IWS uses a different consistency-training objective. A stochastic

predictor can replace P_ϕ when multiple futures matter. Distance in a learned latent space is not inherently a calibrated physical distance.

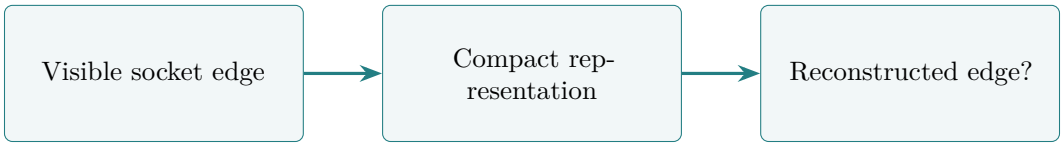


Figure 4.4: Compression can erase the decision. Inspect whether the representation preserves the small geometric feature that separates a safe push from a jam (Wang et al., 2026b).

See Wang et al. (2026b), Figure 2.

Interactive World Simulator trains a compact image autoencoder with a consistency decoder, freezes it, and learns action-conditioned latent consistency dynamics. Its task-specific setup includes default 128×128 imagery and restricted action spaces for some tasks. Interactive demonstrations exceeding ten minutes and the quantitative 192-step, 19.2-second prediction evaluation are distinct evidence. Its Dreamer 4 comparison uses a community implementation trained from scratch (Wang et al., 2026b, Secs. III-B and IV-A–B). These conditions matter before transferring a speed claim to another model or task.

Our insertion question is whether the representation preserves the socket rim, connector orientation, contact progression, and any available force cue. A reconstruction can look acceptable while smoothing away the one edge that distinguishes "push" from "retract." Before training a policy, inspect encoder-decoder reconstructions around this decision. If the distinction is already absent there, a more accurate latent predictor cannot recover it from the compressed input alone.

4.4.1 History and multiple views solve different ambiguities

A wrist camera may see the socket but lose it during approach. A fixed camera may retain the scene but miss insertion depth. Recent history can reveal the approach direction; multiple synchronized views can expose different parts of the same state. Neither guarantees that hidden contact force is observable.

WEAVER and Ctrl-World provide multiview and memory references already developed in Chapter 3 (Guo et al., 2026b; Jain et al., 2026). Here the useful comparison is functional: what state ambiguity does each extra view or retrieved frame resolve? A past image of an unobstructed socket can preserve geometry, but it should not be mistaken for evidence that the current connector has seated. Recorded and generated memory also need distinct provenance: an imagined frame is a prediction carried forward, not a new measurement that corrects drift.

A proposed training record should therefore carry camera identity, timestamp, calibration version where applicable, and a validity mask. Replacing a missing wrist frame with a repeated image without marking it missing changes the apparent dynamics. Likewise, permuting left and right cameras can look like a geometry failure while being a data-interface error.

4.4.2 Measure the entire cycle

For a sequential iteration, let T_{policy}, T_{dynamics}, T_{decode}, T_{judge}, and T_{transfer} be the elapsed times for policy sampling, prediction, image decoding, scoring, and data transfer. The total is

$$T_{\text{cycle}} = T_{\text{policy}} + T_{\text{dynamics}} + T_{\text{decode}} + T_{\text{judge}} + T_{\text{transfer}}. \tag{4.8}$$

These are elapsed costs for a stated batch and horizon, not universal constants. A pipelined implementation needs separate timing of its critical path. If a policy consumes red–green–blue (RGB) images, latent prediction speed alone omits decoding. If evaluation inspects several cameras,

scoring cost may grow too. Report hardware, batch, resolution, views, physical horizon, sampling evaluations, and which terms were timed. Unmatched throughput numbers for IWS, WEAVER, and Ctrl-World are not comparable. The goal is enough useful predictions for a decision, not a large isolated frames-per-second number.

4.5 Make contact error visible

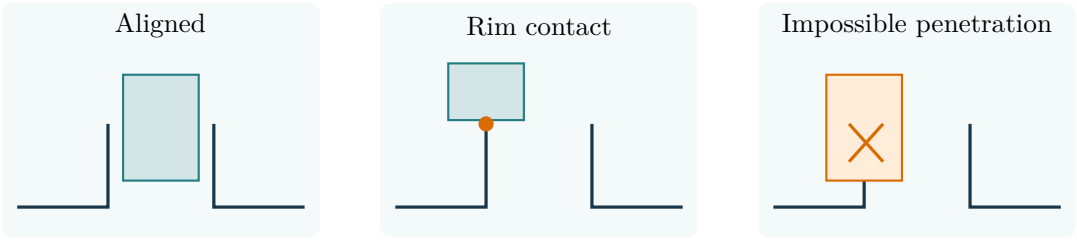
A free-space prediction can tolerate a small pose error and still tell a useful story. At the socket rim, the same error may put the connector on the wrong side of a contact boundary. The model then predicts an entirely different sequence: sliding, sticking, or passing through geometry. Visually plausible interpolation does not enforce nonpenetration.

A mechanical model makes some of the hidden quantities explicit. For a simple mechanical system in Cartesian coordinates, let $\mathbf{x}_t$ and $\mathbf{v}_t$ be positions and velocities, and $\mathbf{M}$ a constant mass matrix. Let $\mathbf{f}_{\text{int}}$ and $\mathbf{f}_{\text{ext}}$ be internal and applied forces. The contact Jacobian $\mathbf{J}_{\text{c}}$ maps velocity to contact coordinates; its transpose maps the contact force $\mathbf{f}_{\text{c},t}$ back to the motion coordinates. A schematic discrete balance is

$$\mathbf{M}\frac{\mathbf{v}_{t+1}-\mathbf{v}_t}{\Delta t} = \mathbf{f}_{\text{int}}(\mathbf{x}_t,\mathbf{v}_t) + \mathbf{f}_{\text{ext}}(a_t) + \mathbf{J}_{\text{c}}(\mathbf{x}_t)^{\top}\mathbf{f}_{\text{c},t}. \tag{4.9}$$

This balance omits details such as the integration of positions and the solution for contact forces; it is not a complete contact solver. For ideal unilateral contact, define $\text{gap}(\mathbf{x})$ as the normal separation and f_{n} as the compressive normal force. The constraints $\text{gap}(\mathbf{x}) \geq 0$, $f_{\text{n}} \geq 0$, and $\text{gap}(\mathbf{x})f_{\text{n}} = 0$ express nonpenetration and the absence of normal force when surfaces are separated. Compliance and friction require additional modeling. The MuJoCo documentation explains why hard and soft contact formulations make different approximations (MuJoCo Developers, 2026).

A learned predictor need not contain a finite element method (FEM) solver. The mechanical view instead identifies what its transition map must represent: geometry, material response, boundary conditions, and contact. Predicting a finite-time state change directly does not automatically preserve the constraints or conservation properties of a numerical solver. Learning from a simulator also does not establish that the simulator's materials match the physical connector.



A small alignment change can change the contact event.
Rendering a clean image does not enforce nonpenetration.

Figure 4.5: Three geometric outcomes. A small alignment error can change the contact event. Explicit state prediction exposes geometric quantities for checking; it also requires appropriate geometry and state information (Wang et al., 2026a).

WorldContact predicts explicit vertex states for deformable manipulation. Its tokenizer distinguishes spatial, topological, controllable, and environmental contacts. Training supervises positions and velocities across rollouts; a differentiable topology projection supports longer predictions. Rendering follows state prediction. Its reported state-rollout acceleration excludes rendering and disk input/output. The real bag-lifting comparison is 13/20 versus 19/20 single attempts with different training-data sets (Wang et al., 2026a, Secs. III–V). This is a useful contrasting representation, not a drop-in RGB model or evidence of connector performance.

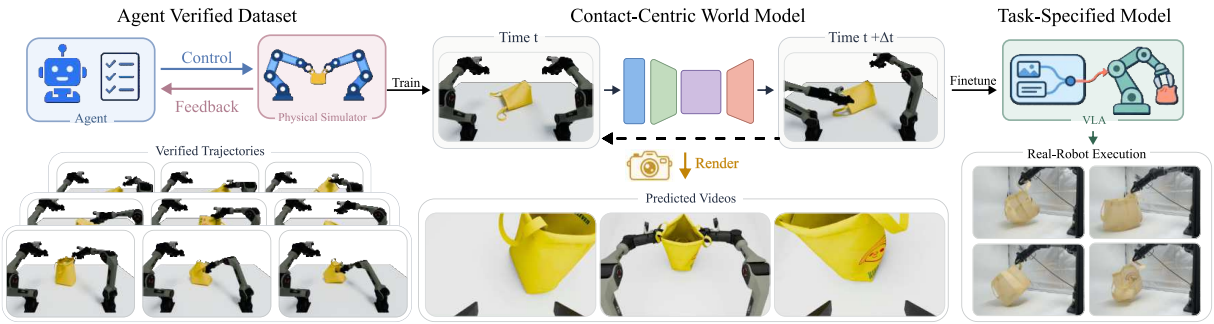


Figure 4.6: State prediction precedes rendering. Figure 1 from Wang et al. (2026a). CC BY 4.0.
Reading the paper figure. Locate the explicit state trajectory and the later rendering step. The experiments concern deformable objects. State supervision makes some physical errors directly measurable.

For our task, explicit geometry could expose penetration depth or connector-axis misalignment directly. The price is obtaining the state, geometry, material assumptions, and boundary conditions needed to initialize and condition the predictor. A visual latent model begins from more readily available observations but makes such physical checks less direct. A hybrid could predict visual features and selected contact-relevant quantities, provided those quantities have credible supervision. That is a design hypothesis to test.

Table 4.1: Our representation comparison for the illustrative insertion task. Required information determines which option is feasible.

Representation	Useful question	Missing information to resolve
Visual latent	Does the approach and visible outcome follow the action?	Does compression retain the rim and depth cues? Can force be inferred?
Explicit state	Are pose, velocity and geometric contact plausible?	How are state, shape and material properties obtained?
Hybrid	Can a visual rollout retain measurable task variables?	Which labels align the visual and physical predictions?

4.6 Turn the model into a reproducible data contract

We now have enough structure to specify one training example. Start from a recorded insertion attempt, align its observations and applied controls, and select a history window. Store the subsequent observations as targets. Keep the success label separate: a failed insertion is still valid evidence of how the world responded. A human correction supplies a new command or label; it does not retroactively change the recorded future.

Table 4.2: Minimum proposed record for the connector example. Use it to audit a new dataset before fitting a predictor.

Field group	What must be recoverable
Identity	Episode, source, task, object/fixture configuration and dataset split.
Measurements	Camera identity and time, proprioception, available force/depth, validity masks.
Controls	Applied sequence, units, frame, duration, controller version, clipping and interruption where logged.
Targets	Observed next frames/state; outcome label and its evidence; uncertain cases retained explicitly.
Generated branch	Parent real context, world-model version, sampled controls, random seed and evaluator version.

Split by source episode before generating overlapping windows. If a held-out episode produces synthetic descendants, those descendants must not leak into training. For a stronger generalization claim, also hold out the relevant object or fixture configurations. Context demonstrations must not reveal the target episode's future outcome. These rules define what the evaluation asks: memorizing nearby frames is different from predicting a new contact configuration.

Training against observed histories and rolling out against generated histories also create different inputs. In a rollout, the model consumes its own mistakes. Testing several horizons, marking generated memory, and restarting from independent real contexts help locate this drift. They do not make arbitrary long predictions reliable. The horizon should be chosen by the decision being supported and the evidence available, rather than by how long an attractive demo can run.

A controllable predictor now has meaningful actions, available conditioning information, an appropriate representation, and traceable targets. To improve a policy with it, we must also define rewards, value targets, update boundaries, and independent evaluation. Chapter 5 develops these requirements and the tests needed to connect imagined success to measured improvement.

4.7 Exercises

Exercise 4.1. A command 0.002 is interpreted as metres per second and applied for 0.1 seconds. What displacement does it request? How does that differ from interpreting it as a displacement in metres?

Exercise 4.2. A transition encoder receives the next image during training. Give one valid use of its output and one invalid deployment use. Explain which missing information causes the problem.

Chapter 5

From Imagined Experience to Evidence of Improvement

Return to the connector paused at the socket rim. Our model can now generate consequences for a specified action. We might try a small push, a slight rotation, or a withdrawal followed by realignment. Generating these alternatives is only the beginning. The learning system still needs a definition of progress, a way to estimate consequences beyond the short rollout, and an update rule that improves the policy without turning a model's mistake into a habit.

This chapter follows that chain. We first separate the learning stages, then build an imagined trajectory and its targets, compare update mechanisms, and examine unreliable feedback. Finally we ask what outside evidence could reject the claim that the policy improved.

5.1 A predictor becomes an environment through a loop

In recorded-action prediction, the action sequence is already known. In policy-in-the-loop imagination, the next action depends on what the model just generated. If the model invents that the connector has seated, the policy may release it. The resulting imagined trajectory is internally consistent with the invented event, even though the real system would have dropped an unseated part.

Let $\widehat{h}_t$ contain the starting real history followed by the imagined actions and generated observations. Let R_ψ be a task-conditioned reward model with parameters ψ, and $\widehat{r}_t$ its predicted reward. The operator append adds the new action and observation to the history. One imagined step is

$$a_t \sim \pi_\theta(\cdot \mid \widehat{h}_t, g, C), \qquad \widehat{o}_{t+1} \sim p_\phi(\cdot \mid \widehat{h}_t, a_t), \tag{5.1}$$

$$\widehat{r}_t = R_\psi(\widehat{h}_t, a_t, \widehat{o}_{t+1}, g, C), \qquad \widehat{h}_{t+1} = \mathrm{append}(\widehat{h}_t, a_t, \widehat{o}_{t+1}). \tag{5.2}$$

A predicted reward is a task score, not an observed outcome. A separate value function V_ν estimates future return under a specified policy and environment. The symbols θ, ϕ, ψ, and ν retain their policy, dynamics, reward, and critic roles throughout the book. Shared implementations must still specify which parameter blocks each loss updates.

The desired return is defined in the real environment. The imagined return is defined under p_ϕ and R_ψ. Optimization sees the latter directly. If the learned transition or score is wrong in regions the policy visits, maximizing imagined return can increase the gap between the two. Freezing the model prevents it from changing during an update; it does not make it correct.

5.2 Learn the world, learn the task, then improve behavior

Each training stage adds a different kind of evidence. Unlabeled video can supervise temporal prediction. Action-aligned video adds control semantics. Task rewards say which outcomes matter. None of these alone supplies a policy that is better than the behavior in the dataset.

See Hafner et al. (2025), Figure 2.

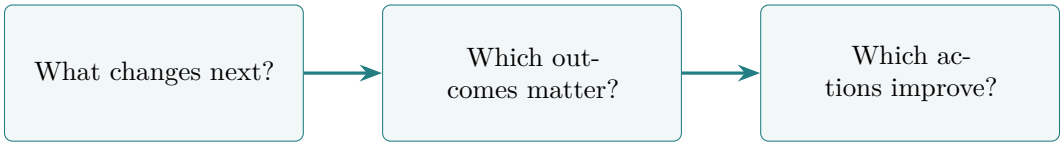


Figure 5.1: Three training questions. Dynamics, task feedback, and policy improvement have distinct targets, even when they share a network (Hafner et al., 2025).

Dreamer 4 gives a concrete staged example: pretrain tokenizer and dynamics; fine-tune with task inputs, policy and reward heads while retaining prediction training; then update policy and value heads on imagined experience with the transformer frozen. Agent tokens attend to other modalities, but the reverse attention is blocked, preventing direct task-token influence on dynamics. Its main policy update uses a variant of preference-based maximum a posteriori policy optimization (PMPO), using advantage signs with a behavioral prior (Abdolmaleki et al., 2024). Shortcut training is a different objective. The principal evidence is Minecraft, not physical manipulation (Hafner et al., 2025, Alg. 1 and Sec. 3.3).

Table 5.1: A teaching ledger for staged learning, informed by Dreamer 4. It distinguishes evidence and update roles without prescribing a robot implementation.

Stage	Evidence or targets	What changes
Representation and dynamics	Observed images, temporal order, available aligned actions	Encoder/decoder first; then predictive dynamics using the learned representation.
Task learning	Actions and task rewards on recorded experience	Policy/reward prediction; retained dynamics training protects the predictive function.
Imagination improvement	Generated trajectories, learned rewards, bootstrapped targets	Policy and value heads in the frozen-transformer recipe.

For insertion, the conceptual benefit is modular diagnosis. If predicted motion ignores commands, study the dynamics and action interface. If plausible failures receive high scores, study reward supervision. If a correct advantage causes the wrong update direction, study optimization. A single aggregate training loss cannot identify which of these failed.

Task-conditioned reward also requires a precise event. “Looks inserted” is not automatically equivalent to electrical connection, a latch engaging, or passing a specified retention test. Before training a reward predictor, choose the event the available evidence can establish. A visual dataset may support a visible-seating label and an uncertain category. A stronger functional success label requires the corresponding measurement. The value function then predicts return for that defined reward, not a more demanding unstated notion of success.

5.3 Build a target before taking a gradient

Suppose fixed world and reward models generate a rollout with H_{roll} transitions. Reset the local step index to $t = 0$ at its start. Let $V_{\bar{\nu}}$ be a fixed critic used to build targets. As before, γ discounts one decision interval and $\lambda \in [0, 1]$ weights longer returns. Define the continuation mask $m_t = 1 - d_t$, where d_t identifies a terminal transition under the task definition. In imagination, this terminal label is also a model estimate and must be checked. The bootstrapped imagined

return target is $\widehat{G}_t^{(\lambda)}$, and its advantage estimate is $\widehat{\mathcal{A}}_t$:

$$\widehat{G}_{H_{\text{roll}}}^{(\lambda)} = V_{\bar{\nu}}(\widehat{h}_{H_{\text{roll}}}, g, C),$$
$$\widehat{G}_t^{(\lambda)} = \widehat{r}_t + \gamma m_t \left[(1-\lambda) V_{\bar{\nu}}(\widehat{h}_{t+1}, g, C) + \lambda \widehat{G}_{t+1}^{(\lambda)}\right], \tag{5.3}$$
$$\widehat{\mathcal{A}}_t = \operatorname{sg}\left(\widehat{G}_t^{(\lambda)} - V_{\bar{\nu}}(\widehat{h}_t, g, C)\right). \tag{5.4}$$

The recursion runs backward for $t = H_{\text{roll}} - 1, \ldots, 0$ from the rollout boundary. At a true terminal transition, $m_t = 0$ removes continuation value. A computational cutoff is a truncation, so the boundary critic estimates the unobserved continuation. This is the book's indexing convention, not a transcription of Dreamer 4's equation. All targets are fixed during a policy update.

Worked example: Delayed success through a short window

Consider two imagined transitions with rewards $\widehat{r}_0 = 0$ and $\widehat{r}_1 = 1$, where the second transition is terminal. For illustration choose $\gamma = 0.9$, $\lambda = 1$, and $V_{\bar{\nu}}(\widehat{h}_0, g, C) = 0.4$. Then $\widehat{G}_1^{(\lambda)} = 1$, $\widehat{G}_0^{(\lambda)} = 0.9$, and $\widehat{\mathcal{A}}_0 = 0.5$. The first movement receives positive credit even though its immediate reward was zero. If the final success was hallucinated, the arithmetic still returns 0.5. Correct credit assignment cannot rescue incorrect evidence.

Let $\mathbb{E}_t$ average over sampled rollout positions. The critic parameters ν can be fitted to the fixed targets with the illustrative regression loss

$$\mathcal{L}_V(\nu) = \mathbb{E}_t\left[\left(V_\nu(\widehat{h}_t, g, C) - \operatorname{sg}(\widehat{G}_t^{(\lambda)})\right)^2\right]. \tag{5.5}$$

This equation explains the role of value fitting; it does not replace a paper's distributional value loss. The reward model scores one transition. A value predicts accumulated future reward under a policy and transition model. A separate benchmark that asks a vision–language model (VLM) to assess "value" is not automatically training this Bellman-style critic.

5.3.1 Freeze the right objects

For a likelihood-based or flow-loss-surrogate policy update, store the rollout, scores, behavior-policy identity, and advantage targets as data. Differentiate the current policy objective through the current policy parameters. Do not let the policy loss reduce its difficulty by modifying its own target or the reward labels in the same gradient calculation.

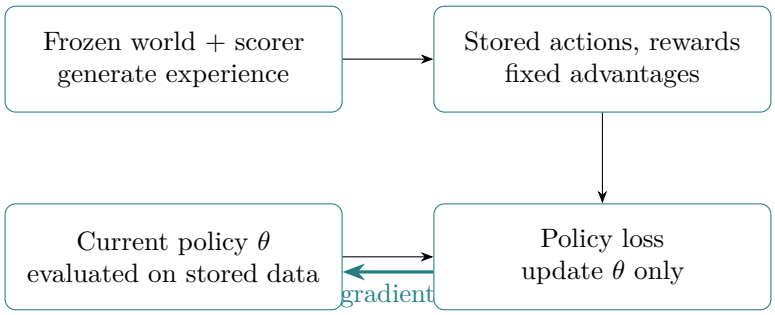


Figure 5.2: A policy update from stored experience. The upper arrows supply fixed data; the emphasized arrow is the policy gradient. Separate critic fitting uses Equation (5.5). This is not differentiable planning through dynamics.

A frozen function can still have derivatives with respect to its inputs. Thus "the world weights are frozen" and "we do not differentiate through the rollout" are different statements. A differentiable

planning method might deliberately propagate input gradients through fixed dynamics. The proposal in Figure 5.2 instead uses sampled experience and detached targets. This distinction matters when translating the story into autodiff code.

5.4 Choose an optimizer without merging different algorithms

The choice of world model does not determine the policy objective. Four mechanisms can use scored trajectories while learning in different ways.

Table 5.2: Policy-update choices. These are alternatives to compare, not interchangeable names for one algorithm.

Mechanism	How feedback enters	Question before adoption
Success filtering	Accept trajectories, then imitate their actions	How many accepted examples are false successes? What useful failures are discarded?
Advantage conditioning	Add a quality/advantage condition to supervised action learning	How are labels estimated, and which condition is selected at inference?
Group-relative RL	Normalize returns within comparable rollout groups and optimize a policy objective	Are groups comparable and policy probabilities tractable under the chosen parameterization?
Flow policy optimization	Use a specified flow-loss surrogate with fixed advantages	Are noise coupling, normalization, clipping and behavior versions correct?

RISE keeps its learned dynamics and value environment fixed during its self-improvement stage and uses advantage-conditioned flow training. VLAW refreshes its world model with real policy experience and uses filtered imagined successes in its policy-improvement process (Guo et al., 2026a; Yang et al., 2026). The first highlights a stable training environment; the second highlights distribution change and model refresh. Neither makes the entire combined loop an FPO algorithm. Chapter 3 gives their fuller treatment.

For an FPO-style candidate, recall the Chapter 3 surrogate

$$\tilde{\rho}_t(\theta) = \exp\left(\widehat{\mathcal{L}}_{\text{old},t} - \widehat{\mathcal{L}}_{\theta,t}\right), \tag{5.6}$$

with old and current losses computed under the specified shared noise/time convention and normalization (McAllister et al., 2025a). This is an exponentiated loss difference, not an exact action-density ratio. The clipped objective can use $\tilde{\rho}_t$ and fixed $\widehat{\mathcal{A}}_t$, but it inherits the surrogate's assumptions. Reducing the current flow loss tends locally to increase this surrogate ratio; that statement does not establish increased real return.

A simple check is that identical parameters and matched noise samples give ratio one. Another is that changing a sum over action coordinates to a mean changes the exponent and therefore the update scale. Such checks establish implementation semantics. They are not evidence that FPO beats filtering on connector insertion. Dreamer 4's PMPO policy update, its shortcut dynamics loss, HaWMPO's group-relative update, and FPO must retain their separate meanings.

5.5 When the model is wrong, which signal should change?

Suppose an imagined push makes the connector pass through the socket wall. The reward model sees a seated connector and returns a high score. We would like to reduce the influence of

that trajectory. But now consider a harmless change in cable shape that differs from the one recorded reference. It may have a large visual prediction error without making the proposed action physically bad.

These cases separate three questions: whether the future is well predicted, whether the action is desirable, and whether the predicted outcome can be verified. A reliability estimate addresses the first or third according to its supervision. It is not automatically a negative action advantage.

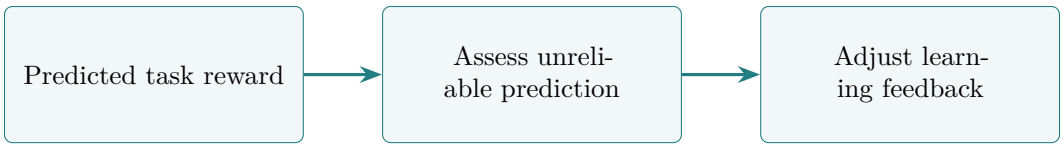


Figure 5.3: A score for success needs a score for reliability. The reliability assessor is another fallible learned component. Its supervision and errors need their own evaluation (Chen et al., 2026).

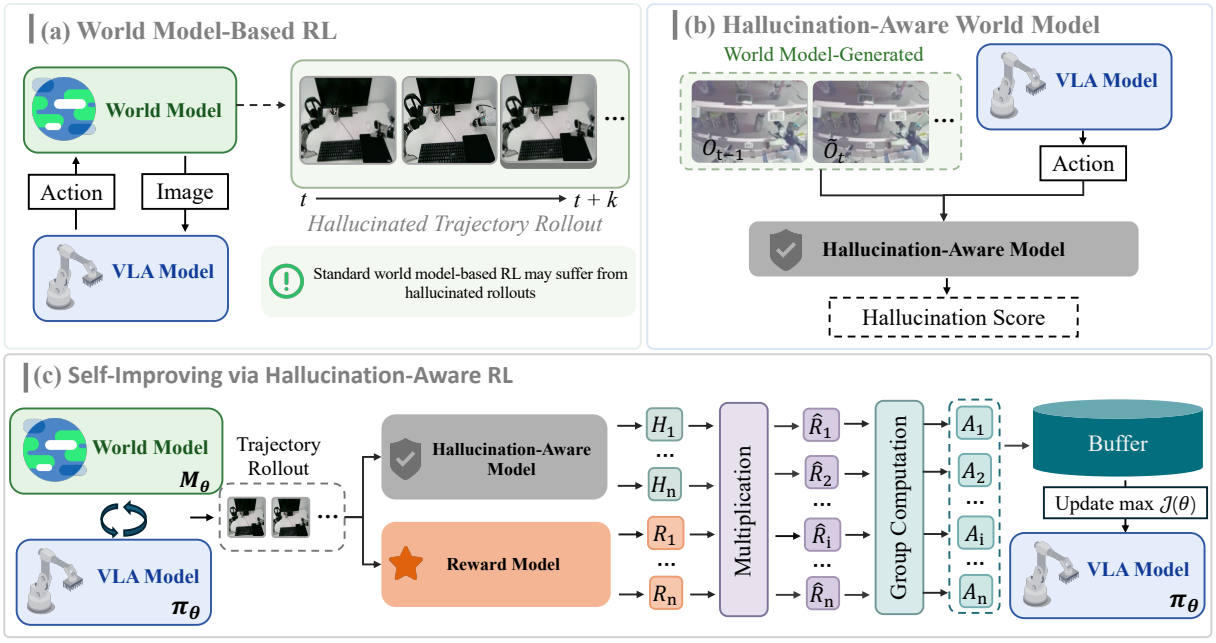


Figure 5.4: Task reward meets a reliability estimate. Figure 1 from Chen et al. (2026). CC BY 4.0.
Reading the paper figure. Follow task reward and hallucination assessment into the policy-learning feedback. In this source diagram, H denotes the role of our u_{hall}, R our $\hat{r}$, and adjusted $\widehat{R}$ our $\tilde{r}$. Its $\widehat{A}$ is an estimated advantage, written $\widehat{\mathcal{A}}$ here; its world-model M_θ plays the role of our p_ϕ.

HaWMPO learns a hallucination assessor from generated-versus-observed futures using feature, depth, motion, and image-quality proxies. At rollout time it uses generated frames, actions, and context without the observed future. Its assessor audit uses 50 LIBERO chunks; physical evaluation covers two tasks with 20 trials per method per task (Chen et al., 2026, Secs. 3 and 4.3–4.6). These proxy labels and small samples limit the evidence for reliability.

In the book's notation, let $u_{\text{hall}} \in [0, 1]$ be the estimated hallucination severity and χ the penalty strength. Let $\hat{r}$ be the raw predicted chunk reward and $\tilde{r}$ the adjusted reward. HaWMPO's reward-softening rule is

$$\tilde{r} = (1 - \chi u_{\text{hall}})\hat{r}. \tag{5.7}$$

These adjusted rewards enter group-relative policy optimization (GRPO), which compares rewards within rollout groups. Figure 5.3 separates the task score from the reliability estimate.

For nonnegative $\hat{r}$ and $\chi \in [0, 1]$, the rule attenuates reward without reversing its sign. Applied to negative rewards, it makes them less negative. This algebraic property matters before transferring

the rule to a new reward convention. The assessor estimates a learned proxy; it cannot certify physical validity.

Worked example: Confidence is not an outcome

Suppose two candidate insertion chunks each receive $\widehat{r} = 1$. With illustrative $\chi = 0.5$, severity estimates $u_{\text{hall}} = 0.2$ and $u_{\text{hall}} = 0.8$ give adjusted rewards 0.9 and 0.6. All else equal, the feedback favors the first chunk. Neither number establishes that the connector seated. A checker blind to hidden latch failure can accept both.

Audit appearance and physical validity separately: include a sharp image of impossible contact, a blurry but valid withdrawal, an occluded result, and a plausible alternative cable configuration. Select thresholds and penalty strengths on a development set. Test on held-out episodes with a stated label definition. Agreement with independent failure labels is stronger evidence than correlation with the proxy used to train the checker.

5.5.1 Keep uncertainty attached to the thing it describes

Variation across sampled futures measures variation under the learned generator. It may reflect real ambiguity, model uncertainty, or sampling artifacts. Agreement across models can still hide shared bias. An interval around a measured success proportion reflects trial sampling under a particular protocol. None is a universal confidence score for deployment.

For insertion, uncertainty about a hidden latch suggests requesting an observation or limiting the claim, not declaring the action a failure. A large predicted penetration error suggests a problem with the generated transition. A verified physical jam is evidence about the executed behavior. Keeping these records separate lets the next training cycle address the correct cause.

5.6 Prediction, decision quality, and improvement need different tests

The most useful external test begins with a question the model could get wrong. Fix a recorded initial history and its applied actions, generate the future, and compare with independently observed consequences. This removes the policy's freedom to change the test while we measure prediction.

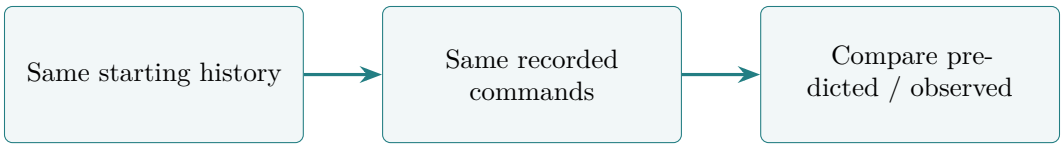


Figure 5.5: Hold the action fixed. This comparison tests consequences. It does not let a new policy choose different actions or establish policy improvement (Duan et al., 2026).

See Duan et al. (2026), Figure 1.

WorldOracle evaluates recorded-action futures against real executions along outcome correctness, spatial alignment, interaction-region fidelity, and multiview action coherence. Its outcome score uses a VLM; the multiview metric aggregates reference-grounded arm alignment rather than directly proving 3D consistency. Limited task/embodiment coverage, single-reference ambiguity, and unequal adaptation conditions constrain comparisons (Duan et al., 2026, Sec. 3 and App. F). Its paired evidence can expose false predicted success, but does not by itself establish closed-loop policy improvement.

5.6.1 Test the distinction the controller cares about

For our connector, average image quality is insufficient. A useful prediction test might ask whether a fixed push produces visible insertion, a rim collision, or withdrawal, with "unobservable" available when the recording cannot decide. Compare near-contact and free-space segments separately. Retain failures in the denominator. If multiple views disagree about the same event, inspect synchronization and visibility before blaming the model.

One observed future is also only one realization. If a cable can settle in two physically valid ways, an alternative prediction should not be called impossible solely because it differs from that recording. Conversely, passing through a solid rim is not rescued by stochasticity. Task-relevant labels and geometric evidence help distinguish plausible diversity from a wrong consequence.

Prediction quality matters for decisions when it helps choose an action. Let $\mathcal{U}$ be a fixed candidate-action set, $\widehat{Q}(h, a)$ a model-based score, and $Q_{\text{ref}}(h, a)$ an independently estimated outcome value under the stated continuation rule. Let $a^\star$ maximize the model score and $a^\star_{\text{ref}}$ maximize the reference value. Use arg max to select a maximizing candidate. Define selection regret as the reference value lost by choosing $a^\star$:

$$a^\star = \arg\max_{a \in \mathcal{U}} \widehat{Q}(h, a), \qquad \text{regret} = Q_{\text{ref}}(h, a^\star_{\text{ref}}) - Q_{\text{ref}}(h, a^\star). \tag{5.8}$$

Fix a tie rule when several candidates have the same score. The candidates can also be action chunks, provided their duration, scoring horizon, and continuation policy are defined consistently. A single recorded action does not reveal the outcomes of all alternatives. Without reference evidence for those alternatives, regret cannot be reported as a measured result.

Finally, policy improvement requires comparing closed-loop behavior. A candidate policy reacts to its own observations and therefore visits different states from its baseline. Prediction on the old policy's recordings does not establish performance on those new states. A later matched physical evaluation should specify task configurations, reset rules, time limits, interventions, success criteria, and trial counts. Repeated attempts and excluded failures must remain visible.

Table 5.3: Three claims and the evidence needed for each. Passing one row does not establish the next.

Claim	Appropriate evidence	What remains unresolved
Predicts consequences	Held-out paired futures under recorded actions	Unobserved counterfactual actions and new policy states.
Improves decisions	Candidate ranking or selection against independent outcome evidence	Repeated feedback and state distribution under deployment.
Improves behavior	Matched closed-loop baseline/candidate trials	Generalization beyond the tested tasks and conditions.

5.6.2 Use denominators that preserve the failure

Suppose a checker accepts 20 generated clips and independent review finds four false successes. Here, false discoveries among accepted clips are $4/20 = 20\%$. That is not the false-positive rate among all actual failures; the latter needs the number of actual failures considered by the checker. Also report rejected true successes, or a checker can appear conservative merely by accepting almost nothing.

For policy comparison, suppose a baseline succeeds on 12 of 20 trials and a candidate on 15 of 20. The observed difference is 15 percentage points, with just three additional successes. It is an estimate under the trial protocol, not a precise universal gain. Report uncertainty at the independent trial or configuration level rather than pretending that thousands of video frames

are independent successes. If the same configurations are paired across methods, preserve that pairing in the analysis.

Ablations should isolate the claim: same real-data budget with and without imagined data; same initialization and imagined batch with filtering versus the proposed optimizer; fixed versus refreshed world model under a specified refresh-data budget. Changing data quantity, scorer, and optimizer together may yield a useful system comparison, but cannot identify which component caused the difference.

5.7 One proposed iteration, from evidence back to evidence

One iteration starts from recorded attempts, generates short imagined branches, and trains a candidate policy. Independent evaluation determines whether to keep the update:

1. **Specify.** Record the action/controller contract, observable success event, data splits, baseline and comparison budget.
2. **Check prediction.** Replay held-out recorded controls; inspect contact outcomes and the decision-relevant representation.
3. **Collect imagination.** Pin world, reward, reliability and behavior-policy versions; store generated histories and truncations.
4. **Update.** Fit value targets separately where needed; apply the chosen policy loss with explicit gradient boundaries.
5. **Evaluate.** Measure prediction and evaluator regressions; obtain independent decision or closed-loop evidence appropriate to the claim.
6. **Refresh or reject.** Diagnose failures, add the right observed evidence, and repeat from a retained baseline rather than adopting the candidate solely because its imagined reward rose.

Refreshing modules creates different kinds of staleness. A new evaluator can rescore stored videos but cannot change their physical truth. A new world model changes the imagined environment, so old generated futures do not become samples from it. A new behavior policy changes the action distribution and may require fresh rollouts for the selected update. Versioning is useful because it lets us identify which of these changes occurred.

Start with the available observations, commands, failure recordings, and success measurements. Missing or unreliable evidence may make data repair or scorer calibration more useful than a new policy optimizer.

Chapter 6 turns these choices into experiments and research directions.

5.8 Exercises

Exercise 5.1. A checker accepts 20 clips; 4 are false successes. There were 50 actual failures in the full audit. Compute false discovery proportion and false-positive rate. Which denominator belongs to each?

Exercise 5.2. Three candidates have reference values 0.2, 0.8, 0.5 and model scores 0.9, 0.7, 0.4. Which is selected, and what is its selection regret? Could a single recorded action establish this regret?

Chapter 6

Choosing Your First Research Direction

A policy chooses actions; dynamics predicts consequences; rewards define what matters. Independent evidence tests whether an update improves behavior. Your next step is to choose one failure in this chain and make it measurable.

Start with a system whose inputs, commands, and outcomes you understand. Increase complexity when the simple experiment can no longer answer your question.

6.1 A complete experiment on a laptop

As a controlled warm-up before a paper-level experiment, the repository includes `examples/one_dimensional_world.py`. It models a one-dimensional slider with a command deadband. Here s_t is position in metres and a_t is a requested displacement in metres over a fixed control interval. The setting $\kappa \geq 0$ is a deadband width in metres: commands whose magnitude does not exceed it produce no motion. Define $\operatorname{sign}(a)$ to be -1, 0, or 1 according to whether a is negative, zero, or positive. The transition is

$$s_{t+1} = s_t + \operatorname{sign}(a_t) \max(|a_t| - \kappa, 0). \tag{6.1}$$

This is a deliberately simple response law. It has no velocity state, inertia, collisions, or faithful friction/contact solver. Position is sufficient for its transition because we defined a system with no hidden velocity dependence. The purpose is to study model error and decisions under a known transition, not to simulate a real contact mechanism.

The experiment generates 400 independent one-step training examples with positions between 0.1 and 0.9 metres, actions between -0.1 and 0.1 metres, and $\kappa = 0.02$ metres. Training next-position measurements receive independent uniform noise between -0.003 and 0.003 metres. The dynamics used for evaluation are noise-free, so model error can be measured against known state rather than noisy observations. This privileged reference is available because the system is synthetic.

Three predictors use the same training budget. Persistence predicts no movement. A linear predictor fits one gain multiplying the action. A deadband predictor searches a prespecified grid of possible widths, from 0 to 0.06 metres in increments of 0.001 metres, and selects the width with lowest training squared error. It has the correct family of response laws for this toy world. That favorable modeling assumption must be stated when interpreting its result.

Test the fitted predictors on two separately generated sets of 200 examples: the original width 0.02 and a shifted width 0.04. The training data contain only the original width. Test examples use distinct random-number streams, and neither test set selects the fitted parameter. The second test therefore asks about a specified change in physical conditions, not merely new random examples from the same condition.

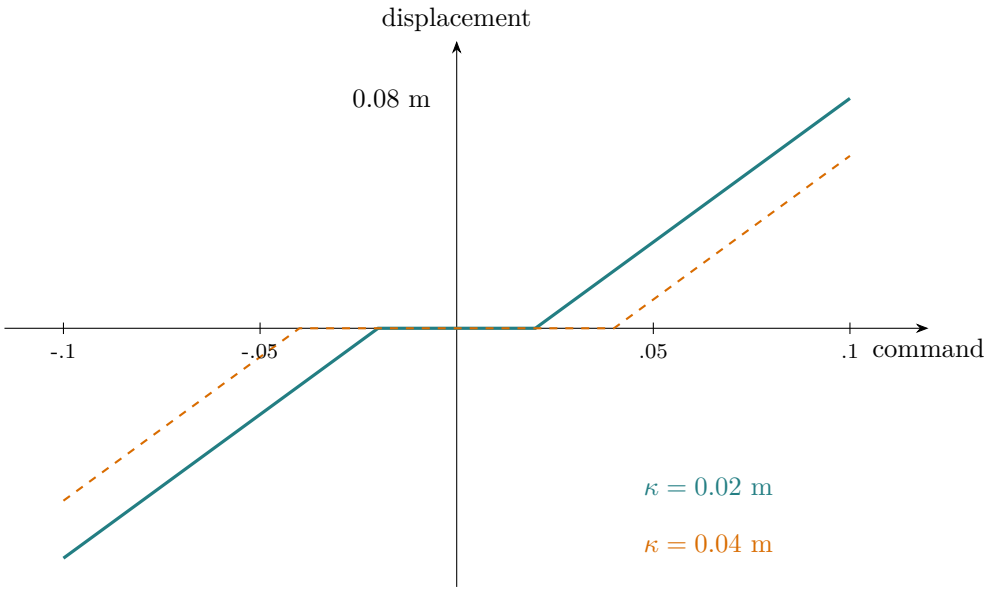


Figure 6.1: The toy action response. Small commands leave the slider still; larger commands move it after subtracting the deadband. Increasing the deadband changes the action consequence while preserving the same command units.

6.1.1 Prediction and decision metrics

The first metric is mean absolute next-position error, in metres. The second asks which action the model chooses. For each initial position, a target g is sampled within 0.06 metres. The model evaluates the same 41 candidate actions from -0.1 to 0.1 metres in increments of 0.005 metres and selects the one with smallest predicted distance to the target. Ties prefer smaller command magnitude, then the smaller signed command. All predictors use this rule.

For this one-step task, reward is negative absolute distance to the target after execution. A decision succeeds if that distance is at most 0.004 metres. Reference values are computed by applying every candidate to the known toy transition. Selection regret is the reference value of the best candidate minus that of the selected candidate, as in Equation (5.8). We can measure this counterfactual quantity here because the simulator supplies outcomes for all candidates. An ordinary recording of one action would not.

The program writes the seed, data counts, settings, fitted parameters, prediction error, decision regret, and success counts. It does not retrain on the shifted test set. Run it from the repository root:

```
python3 examples/one_dimensional_world.py -output results/toy
```

6.1.2 Read the failure before changing the model

First compare persistence with the fitted predictors. Next compare the original and shifted conditions for each predictor. A fitted width close to 0.02 is expected to describe the original condition better than 0.04. A loss of decision quality under the shift is evidence that the assumed response law or condition has become inaccurate. It does not, by itself, motivate a larger policy optimizer.

Now consider what would let the model adapt. If the current width is measured, include it as a dynamics input and train across several widths. If it is hidden but constant within an episode, earlier action–response pairs may help estimate it. These pairs would form physical context C_{dyn}, distinct from the demonstration context C that specifies the task. If the width changes without

any observable clue, the model should represent uncertainty or obtain new evidence; it cannot infer an arbitrary hidden change with certainty.

The current example consists of independent one-step cases. Studying identification from history requires a new multi-step dataset with condition-consistent episodes and a split that keeps episodes separate. Merely passing random old examples into a history encoder would not establish that they describe the present slider.

6.2 Match a direction to a question

Choose the question first, then the smallest experiment that could answer it. The following directions extend the book's running problems.

Table 6.1: Research entry points with a measurable first result.

Interest	First question	Evidence to collect
Graphics and geometry	Does compression erase the feature that changes the action choice?	Paired reconstructions and action ranking near small alignment changes.
Control and dynamics	Can short interaction history identify a hidden response setting?	Held-out episodes with known settings, including changes between episodes.
Generative modeling	Does a stochastic predictor preserve several valid futures without accepting impossible ones?	Multiple reference realizations and independent physical constraints.
Robot learning	Does imagined data improve a fixed baseline at a matched observed-data budget?	Baseline/candidate behavior on held-out configurations, with all attempts counted.
Evaluation	Can a scorer distinguish visible success from verified functional success?	Independently labeled outcomes, uncertain cases, and full confusion counts.
Optimization	Does an update help when model and scorer are held fixed?	Matched training data and budget, numerical checks, and external behavior evaluation.

For a graphics-oriented reader, the first direction is a natural bridge. Begin with known geometry and a few camera positions. Introduce a small alignment change that reverses the preferred action. Compare whether images, reconstructed geometry, and compressed latents preserve that change. A lower image loss is useful evidence only if the relevant distinction survives. NeRF and Gaussian Splatting provide representation references, while Chapter 4 explains why contact requires an additional transition model (Kerbl et al., 2023; Mildenhall et al., 2020).

For a control-oriented reader, the toy deadband is a starting point for system identification: inferring the response law from inputs and outputs. Ask how many interactions distinguish two settings, whether the chosen actions reveal the difference, and whether estimation improves later decisions. A useful action can both pursue the goal and expose uncertainty. Establish this in a transparent simulator before using a costly visual predictor.

For an evaluation-oriented reader, construct a small set of outcomes whose labels require different evidence. A cup can look held while resting on a support; a connector can look seated without electrical continuity. State what each sensor can establish. Evaluate false accepted successes and missed true successes separately. The result may be a clear measurement limitation rather than a new model, and that limitation can prevent misleading learning claims.

Only choose the optimization direction after the targets are credible. If every failed grasp is labeled successful, a better optimizer may learn the wrong behavior faster. Compare a simple filtering baseline with a more complex update under the same data and scorer. If the complex

method helps, investigate which change caused it; do not change the dynamics, data budget, and optimizer simultaneously and attribute the difference to the optimizer alone.

6.3 Move from a toy result to a paper-level experiment

The slider makes a failure interpretable because its transition law is known. A research model removes that convenience: its representation, action adapter, uncertainty, and evaluator can all fail at once. The next experiment should isolate one of these changes. Moving directly from the slider to a large pretrained model would change too many assumptions to explain the result.

6.3.1 Representation is a hypothesis about sufficient information

Compare the visual-latent route in IWS and WEAVER with explicit state prediction in World-Contact (Jain et al., 2026; Wang et al., 2026a,b). These are not interchangeable implementations of the same input contract. A visual predictor begins with measured images; an explicit-state predictor needs the geometry and state information required by its representation. A meaningful comparison first identifies what is available at deployment and what is supplied only by training supervision.

A controlled study could fix recorded actions and a set of near-contact decisions, then evaluate a visual latent with and without additional geometric supervision. Measure both the retained geometric distinction and the resulting action ranking. Keep additional labels and compute visible in the comparison. If geometry helps only when an unavailable simulator state is supplied at deployment, the result supports that privileged setting rather than a camera-only controller.

6.3.2 Context must explain either intention or physical response

SynthICL's demonstration context C explains the intended task, while physical context C_{dyn} would supply evidence about the transition law (Qian et al., 2026a). These roles can coexist, but they require different supervision. A demonstration placing a cup on a different coaster changes the desired behavior. An earlier push revealing a slippery surface changes the estimated consequence of a fixed command. Neither should be silently substituted for the other.

A study of physical context can fix the task and action convention, vary physical settings across episodes, and ask whether a short observed interaction improves predictions in the same episode. Separate interpolation among trained settings from extrapolation to unseen settings. Compare against a predictor without context and a reference given the true setting. The reference measures the value of missing information; it is not a deployable baseline if that information is unavailable. A context encoder that memorizes scene appearance may perform well on a random frame split and fail on a new material or controller.

6.3.3 A learning-loop comparison needs matched evidence

RISE, VLAW, and FPO answer different parts of the learning problem. RISE illustrates advantage-conditioned improvement inside a fixed learned environment; VLAW refreshes the model with real experience and uses success-filtered policy training; FPO specifies a flow-loss-based policy update (Guo et al., 2026a; McAllister et al., 2025a; Yang et al., 2026). Treating their published headline results as an optimizer ranking would mix task, model, data, and feedback differences.

For a comparison of update rules, begin with the same policy initialization, observed-data budget, frozen dynamics, and evaluator. Specify whether the methods receive the same stored trajectories or each collects fresh ones: the latter changes both the data distribution and the

update. A fixed-data comparison tests what an update extracts from given experience; an iterative comparison tests the entire collection-and-learning loop. Report both only if both are actually evaluated.

For FPO, reproduce the noise/time convention, dimensional reduction, clipping, and behavior-policy snapshots before interpreting a result. Then compare with a simpler filtering or conditioning baseline using independent outcomes. If the candidate improves only the same evaluator that generated its training labels, the central improvement claim remains unresolved. Appendix A explains why a plausible loss-ratio derivation cannot close that evidence gap.

6.3.4 Read a figure as an experimental specification

An architecture figure is a compact set of claims about information flow. For every arrow, identify its tensor, timing, units where applicable, and availability during training and deployment. For every box, identify the parameters updated by each objective. For a result plot, identify the independent unit, baseline, data budget, and uncertainty. Our flowcharts summarize the argument; the reproduced paper figures let you check it against the authors' actual presentation.

End a paper reproduction with an explicit boundary: which equation, component, or protocol was reproduced, and which was replaced by a teaching approximation? A useful outcome may be that the reported ordering changes under a new action convention or evaluator. That is a research direction only after the comparison rules out implementation mistakes and unsupported changes in the question.

6.4 A two-week first pass

Use days one through three for Chapters 1 and 2. Explain execution, sampling, and training without looking at the text. Run the toy example and verify one transition by hand.

On days four through six, read Chapter 3's world-model and reward sections. Trace one return calculation and a prediction error that changes an action choice. Postpone the optional derivations until reward, value, and advantage are clear.

On days seven through nine, use Chapter 4 to write an action schema and an input-availability table. Choose one extension: condition shift, hidden settings, reduced observations, or representation error. Fix the baseline and budget.

On days ten through twelve, use Chapter 5 to specify the claim, independent unit, split, metric, and failure criterion before running the final test. Inspect individual failures as well as averages.

Use the final two days to report the question, assumptions, method, results with counts and units, a limitation, and the next experiment. Read one relevant primary paper and map its inputs, objective, and evidence onto the same template.

6.5 Exercises

Exercise 6.1. Starting at 0.5 metres with $\kappa = 0.02$ metres, compute the next position for actions 0.01, 0.05, and -0.05 metres. Which of these reveals that the response is not a simple unit-gain displacement?

Exercise 6.2. Propose an extension that estimates a hidden deadband from history. Define an episode, the available inputs, the held-out split, a baseline, and one outcome that would count against your hypothesis.

Appendix A

Flow Losses and Likelihood Bounds

The operational picture is already available: compare old and current flow losses on the same action and noise, exponentiate the difference, and weight the resulting clipped surrogate by advantage. The remaining question is why a prediction loss should be relevant to likelihood at all. This appendix supplies that connection. It can be read after the main chapters. First follow the meaning of the bound and the cancellation argument; then examine the schedule and weighting assumptions.

A lower bound can be optimized when the quantity above it is inconvenient to calculate. However, raising the bound does not guarantee that the quantity itself rises by the same amount: the gap can change too. That simple observation is the reason we retain the word *surrogate* throughout the FPO discussion. An evidence lower bound, or ELBO, is a tractable variational bound related to log likelihood. Let B_θ denote an evidence lower bound and $\mathcal{K}_\theta$ its nonnegative variational gap. For the compatible probabilistic construction, write

$$B_\theta(\mathbf{a} \mid c) = \log \pi_\theta(\mathbf{a} \mid c) - \mathcal{K}_\theta(\mathbf{a}, c), \qquad \mathcal{K}_\theta \geq 0. \tag{A.1}$$

The nonnegative gap is a Kullback–Leibler (KL) divergence term between latent/path distributions in the variational construction. It is *not* the KL divergence from the old policy to the current policy. FPO's implemented update does not explicitly calculate that gap (McAllister et al., 2025a).

The relation between a denoising loss and this bound depends on the schedule, prediction parameterization, and weighting. To see why an unweighted mean squared error (MSE) is not generally a negative ELBO, let $\sigma \in [0, 1]$ parameterize a forward noising process from data to noise. It is a time coordinate here, not a standard deviation. Let $\ell_{\mathrm{snr}}(\sigma)$ be the decreasing log signal-to-noise ratio along that process. It is unrelated to the GAE parameter λ. Let w be a nonnegative weight function and $\widehat{\epsilon}_\theta$ the model's noise prediction from a corrupted sample, its noise level and c; these inputs are suppressed. Sample σ uniformly and draw standard Gaussian noise ϵ. With the target action and conditioning held fixed, denote a representative weighted loss by $\mathcal{L}^w_\theta$:

$$\mathcal{L}^w_\theta = \frac{1}{2}\mathbb{E}_{\sigma,\epsilon}\left[w(\ell_{\mathrm{snr}}(\sigma))\left(-\frac{d\ell_{\mathrm{snr}}}{d\sigma}\right)\|\widehat{\epsilon}_\theta - \epsilon\|^2\right]. \tag{A.2}$$

Here σ runs in the forward noising direction, opposite to our noise-to-action τ. Even choosing $w = 1$ leaves the schedule derivative and parameterization factors. This is not arbitrary unit-weight velocity MSE. For $w = 1$, under the compatible diffusion assumptions and endpoint treatment used in the derivation, the loss equals a negative clean-data ELBO plus a parameter-independent term. More general monotone weightings, including the optimal-transport flow case discussed in FPO's appendix, connect to an *expected bound for noise-perturbed data*, rather than automatically to the same clean-action bound (Kingma and Gao, 2023; McAllister et al., 2025a).

For a compatible bound B_θ, let b_{const} collect terms independent of the policy parameters. Suppose $\mathcal{L}^w_\theta = -B_\theta + b_{\mathrm{const}}$ with the same forward process and schedule for old and current models. Use the subscript "old" for evaluation at θ_{old}. The common term cancels in the loss difference:

$$\exp(\mathcal{L}^w_{\mathrm{old}} - \mathcal{L}^w_\theta) = \exp(B_\theta - B_{\mathrm{old}}). \tag{A.3}$$

It is therefore unnecessary to calculate an absolute bound to form the difference. But cancellation does not turn a bound ratio into an exact action-likelihood ratio. In the clean-data case,

Equation (A.1) gives

$$\exp(B_\theta - B_{\text{old}}) = \frac{\pi_\theta(\mathbf{a} \mid c)}{\pi_{\theta_{\text{old}}}(\mathbf{a} \mid c)} \exp(\mathcal{K}_{\text{old}} - \mathcal{K}_\theta). \tag{A.4}$$

The gap terms need not cancel.

Suppose an old model has log density -2.0 and gap 0.5, so its bound is -2.5. A new model with log density -2.1 and gap 0.1 has a higher bound, -2.2, despite a slightly lower exact density. The bound has risen while the exact density has fallen. For the noise-perturbed interpretation, it is even more important to retain the precise bound being compared. Lowering a surrogate loss is not a sample-by-sample guarantee of increased exact density or physical return.

Finite Monte Carlo introduces another distinction. Even if a loss average estimates its expectation without bias, exponentiating the difference generally produces a biased estimate of the exponentiated expected difference. More noise/time samples can reduce variance, but cannot convert the surrogate into an exact likelihood calculation. Loss scaling also matters: summing rather than averaging over action dimensions changes the exponent. A policy implementation must record schedule, time sampling, weighting, dimensional reduction, and numerical stabilization together. Those details determine what is being optimized.

Appendix B

Selected Exercise Solutions

These solutions show the essential reasoning. Open-ended designs can have several valid answers if their assumptions and evidence are explicit.

Exercise 1.1. The future position depends on velocity as well as position. A second timestamped image can help estimate velocity; direct velocity measurement can also help. Neither guarantees that other relevant variables, such as contact force, are known.

Exercise 1.2. The gradient remains -1.2, so the new value is $0.2 - 0.5(-1.2) = 0.8$. It fits this one scalar target exactly. There is no evidence about new observations, a different target, or task success.

Exercise 2.1. The executed prefix lasts $3 \times 0.04 = 0.12$ seconds. Twelve is the proposed horizon, three is the executed prefix, and eight is a sampling budget. None specifies how many optimizer updates occur; deployment may use zero.

Exercise 2.2. Hold checkpoint, observations, and task evaluation fixed while changing only the demonstration; a behavior change tests context response. For persistent learning, compare retained checkpoints under matched inputs after training, including removal of the new demonstration and checks on previous tasks. Record the actual parameter update and evaluate behavior independently; a changed file alone does not establish useful learning.

Exercise 3.1. The first-step return is $0 + 0.9(0) + 0.9^2(1) = 0.81$. Subtracting 0.6 gives 0.21. If success is imagined, the arithmetic is unchanged but its evidential meaning changes: the target depends on the world model and reward model being correct.

Exercise 3.2. The surrogate is $\exp(0.8 - 0.6) = \exp(0.2) \approx 1.2214$. It is an exponentiated estimated flow-loss difference. Monte Carlo noise, scaling conventions, and the likelihood-bound gap prevent treating it as an exact density ratio; see Appendix A.

Exercise 4.1. Velocity times duration gives $0.002 \times 0.1 = 0.0002$ metres. Interpreting 0.002 as displacement gives ten times as much requested motion. Both calculations assume the stated interval and ideal command execution.

Exercise 4.2. The future-derived code can be a fixed training target for an adapter using current history and action. It cannot be a required deployment input before that future is observed. If it encodes an outcome not determined by the available inputs, the adapter needs more evidence or uncertainty modeling.

Exercise 5.1. False discoveries among accepted clips are $4/20 = 20\%$. False positives among actual failures are $4/50 = 8\%$. The accepted set and actual-failure set are different denominators. Other confusion counts remain necessary to describe coverage.

Exercise 5.2. The model selects the first candidate. The reference prefers the second. Regret is $0.8 - 0.2 = 0.6$. A recording of one action does not supply all three reference values; additional trials or a justified reference simulator would be needed.

Exercise 6.1. The next positions are 0.5, 0.53, and 0.47 metres. All differ from the unit-gain prediction $s_t + a_t$ because the nonzero commands lose displacement to the deadband. The small command reveals the flat region directly.

Exercise 6.2. One valid design keeps a hidden width fixed within each multi-step episode and varies it between episodes. Inputs are observed positions, applied commands, and timestamps; the true width is used only for evaluation. Split by complete episodes and reserve some widths for a separate generalization test. Compare a fixed training-set width with an estimator using recent action–response pairs under equal interaction budgets. If history lowers parameter error but worsens held-out decision regret, the proposed claim about better decisions is not supported. Define measurement noise and how exploratory actions count toward the budget.

Symbols and Conventions

An unadorned observation is measured; a hat marks a prediction or estimate. A tilde marks a constructed surrogate or adjusted quantity. The subscript old means evaluation at a fixed behavior-policy snapshot. A bar over critic parameters marks a fixed target copy. Bold **a** always means an action chunk, while a means one action, which can itself be a vector. Bold attention and mechanics symbols distinguish matrices or vectors from scalar learning quantities.

Interaction, context, and time

Symbol	Meaning
t	Integer control-step index. A local imagined rollout starts at $t = 0$.
$t_{\text{phys},t},\ \Delta t$	Physical timestamp of step t and duration of one control interval. These have units of time.
$s_t,\ o_t,\ \widehat{o}_t$	Full physical state, measured observation, and predicted observation.
$h_t,\ \widehat{h}_t$	Available observation–action history; an imagined history starts with real evidence and then appends generated transitions.
$a_t,\ \mathbf{a}_t$	One action and a proposed chunk of consecutive actions. Replanning may replace the unexecuted part of a chunk.
$H,\ d_a,\ K$	Number of actions in a chunk, dimension of one action, and number executed before replanning; $K \leq H$.
H_{roll}	Number of transitions in an imagined rollout, distinct from chunk length.
T_{ep}	Terminal step index of an episode, not a duration in seconds.
$g,\ C$	Task specification and optional demonstration context. Context need not include action labels.
$c_t = (h_t, g, C)$	Policy conditioning information. Write c for a generic training example.
C_{dyn}	Additional evidence about physical conditions, such as earlier interactions.
L	Number of recent observation latents supplied to a predictor.
$o_{j:k},\ z_{j:k}$	Sequence from index j through index k, inclusive.
$i,\ N$	Training-example index and number of examples in a batch.
$m,\ M$	Noise/time-sample index and number of samples in a Monte Carlo loss estimate.
$j,\ k$	Local indices: j indexes a sampling grid or candidates; k indexes a future offset. The use is stated locally.

Models and their parameters

Symbol	Meaning
$\pi_\theta,\ \theta$	Policy distribution and its trainable parameters. Its argument determines whether the distribution is over an action or a whole chunk.
θ_{old}	Fixed parameters of the behavior policy that collected the current batch.
v_θ	Policy velocity network used for flow sampling and flow-loss evaluation. It is not the robot's physical velocity.
$p_\phi,\ \phi$	Conditional world-model distribution and its parameters.
P_ϕ	Point-prediction alternative for latent dynamics, rather than a distribution.
$R_\psi,\ \psi$	Learned reward model and its parameters.
$V_\nu,\ \nu,\ \bar{\nu}$	Learned critic, its trainable parameters, and a fixed parameter copy used to form targets.
$q_\zeta,\ \zeta,\ \widehat{y}$	Outcome evaluator, its parameters, and its predicted label. The label is not automatically a calibrated probability.
$E_\alpha,\ D_\beta$	Observation encoder and decoder, with parameters α and β.
z_t	Observation latent $E_\alpha(o_t)$.
$E_{\text{tr}},\ z^\star$	Fixed transition encoder and its future-derived latent target. The star marks a target here, not an optimal action.
$f_\omega,\ \omega,\ \widetilde{z}$	Action adapter, its parameters, and the latent condition predicted before the future is observed.
η_{learn}	Positive learning rate in a gradient-descent update.
$\Pr, \text{sign}$	Probability of an event; sign of a scalar (negative, zero, positive).
$\mathcal{D}$	Dataset of aligned training examples.
$d_{\text{image}},\ d_{\text{latent}}$	Chosen reconstruction and latent-prediction losses. Neither is inherently a physical distance.
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	Attention query, key, and value matrices. These differ from action values Q^π, execution prefix K, and critic values V^π.
d_{key}	Shared dimension of an attention query and key.

Rewards, returns, and policy updates

Symbol	Meaning
$r_t,\ \widehat{r}_t$	Reward for the transition beginning at t, and its model prediction.
γ	Discount factor per stated decision interval, between zero and one.
G_t	Discounted return from step t through the episode.
G_i^{target}	Fixed return target for critic training sample i.
$\widehat{G}_t^{(\lambda)}$	Bootstrapped imagined return target using return-mixing parameter λ.
$V^\pi,\ Q^\pi$	Expected return under policy π; expected return after a specified first action and then following π.
$\mathcal{A}^\pi,\ \widehat{\mathcal{A}}$	True advantage $Q^\pi - V^\pi$ and an estimated advantage.

Symbol	Meaning
$d_t,\ m_t = 1 - d_t$	Terminal indicator and continuation mask. A computational cutoff alone is not termination.
δ_t	One-step temporal-difference residual.
λ	Return/advantage mixing parameter in $[0, 1]$; never log signal-to-noise ratio in this book.
b_{adv}	Advantage-derived bin or indicator used as a policy input.
$J(p),\ p$	Expected return and left-action probability in the two-action teaching example.
$f(a)$	Fixed outcome score in the score-function gradient derivation.
$S,\ S_{\text{PPO}}$	Local empirical policy surrogate and PPO's clipped surrogate, both maximized.
$\rho,\ \tilde{\rho}$	Exact current-to-behavior action-density ratio and FPO's flow-loss surrogate ratio. They are not interchangeable.
ϵ_{clip}	Positive clipping width in the policy surrogate; distinct from Gaussian noise ϵ.
$\mathcal{L},\ \ell_\theta$	A minimized loss and a loss for one sampled noise/time pair. Subscripts identify the objective or parameters.
$\widehat{\mathcal{L}}_{\theta,i},\ \widehat{\mathcal{L}}_{\text{old},i}$	Monte Carlo flow-loss estimates for sample i, under current and fixed old policies.
$\mathcal{V}_{\text{RISE}},\ \widehat{\mathcal{A}}_{\text{RISE}}$	RISE-specific progress/value predictor and chunk score. The latter is not assumed equal to every textbook advantage estimator.
$u_{\text{hall}},\ \chi,\ \widetilde{r}$	Estimated hallucination severity, penalty strength, and adjusted reward.
$\mathcal{U}$	Fixed set of candidate actions for a decision comparison.
$\widehat{Q},\ Q_{\text{ref}}$	Model-based decision score and independently estimated reference outcome value under a stated continuation rule.
$a^\star,\ a^\star_{\text{ref}}$	Candidates maximizing the model score and reference value, respectively.

Generative sampling and likelihood bounds

Symbol	Meaning
$\tau,\ X_\tau$	Artificial flow time from noise (0) to action data (1), and the intermediate action sample.
$\epsilon,\ I$	Standard Gaussian noise and identity covariance of the appropriate dimension.
u_τ	Known reference velocity along a constructed training path.
τ_j	Point on the numerical sampling grid.
σ	Forward noising coordinate from data to noise in Appendix A; not a standard deviation.
$\ell_{\text{snr}}(\sigma)$	Log signal-to-noise ratio along that forward process.
$w,\ \widehat{\epsilon}_\theta$	Nonnegative loss-weighting function and model-predicted noise.

Symbol	Meaning
$\mathcal{L}_\theta^w$	Weighted denoising loss under the stated schedule and parameterization.
$B_\theta,\ \mathcal{K}_\theta$	Variational lower bound on log likelihood and its nonnegative gap, for a compatible probabilistic construction.
b_{const}	A term independent of policy parameters, common to old and current losses in the cancellation argument.

Physical interfaces and computational cost

Symbol	Meaning
$\mathbf{q}_t$	Robot configuration used to determine a coordinate transformation.
$\Delta\mathbf{p}^{\mathrm{T}}, \Delta\mathbf{p}^{\mathrm{B}}$	Tool-frame and base-frame representations of one displacement. Superscripts name frames.
$\mathbf{R}_{\mathrm{BT}}$	Rotation mapping tool-frame vectors into base coordinates.
$F_{\Delta t},\ \kappa$	Physical transition over interval Δt, and controller/environment settings. In Chapter 6, κ is specifically a scalar deadband width.
$\mathcal{O},\ \varepsilon_t^s,\ \varepsilon_t^o$	Observation map, process error, and observation error in the teaching state model.
$\mathbf{x}_t, \mathbf{v}_t, \mathbf{M}$	Cartesian positions, physical velocities, and a constant mass matrix in the simplified mechanics example.
$\mathbf{f}_{\text{int}}, \mathbf{f}_{\text{ext}}$	Internal and applied forces in those coordinates.
$\mathbf{J}_{\mathrm{c}}, \mathbf{f}_{\mathrm{c},t}$	Contact Jacobian and contact force in contact coordinates.
$\mathrm{gap}(\mathbf{x}),\ f_{\mathrm{n}}$	Normal surface separation and compressive normal contact force.
T_{cycle}	Elapsed time of one sequential prediction/decision cycle.
$T_{\text{policy}}, T_{\text{dynamics}}$	Policy-sampling and world-prediction elapsed times.
$T_{\text{decode}}, T_{\text{judge}}, T_{\text{transfer}}$	Decoding, scoring, and data-transfer elapsed times.

Operators and recurring abbreviations

Notation	Meaning
$\sim,\ \lvert,\ \cdot$	Sampled from; conditioned on; an unspecified argument of a distribution.
$\mathbb{R},\ \mathcal{N}(0, I)$	Real numbers and a zero-mean Gaussian with identity covariance.
$\mathbb{E},\ \nabla_\theta$	Expectation over specified random inputs and gradient with respect to θ.
$\lVert\cdot\rVert,\ (\cdot)^\top$	Euclidean norm (flattening an array when needed) and transpose.
sg	Stop-gradient: preserve the value while setting its derivative to zero for the update.
$\mathrm{clip}(x, l, u)$	Restrict scalar x to the interval with lower bound l and upper bound u.

Notation	Meaning
$\arg\max$	Return an argument attaining the maximum; use a specified tie rule.
$\mathrm{softmax}, \mathrm{Attn}$	Normalize scores into nonnegative weights summing to one; scaled dot-product attention.
append	Add a new action and observation to a history.
CFM, ODE, NFE	Conditional flow matching; ordinary differential equation; number of function evaluations.
RL, TD, GAE	Reinforcement learning; temporal difference; generalized advantage estimation.
PMPO	Preference-based maximum a posteriori policy optimization.
PPO, FPO, GRPO	Proximal Policy Optimization; Flow Policy Optimization; group-relative policy optimization.
ELBO, KL, MSE	Evidence lower bound; Kullback–Leibler divergence; mean squared error.
VLA, VLM	Vision–language–action model; vision–language model.
RGB, KV	red–green–blue image channels; attention keys and values.
FEM	Finite element method.
TP, FP, FN, TN	True positives, false positives, false negatives, and true negatives, relative to the stated success label.

The published model names π_0, $\pi_{0.5}$, and $\pi^*_{0.6}$ are names, not time indices or instances of the parameter subscript θ.

Task and context arguments may be suppressed after they are defined, but their meaning does not change. The same rule applies to batch and time indices. An equation using one action can be extended to a chunk only after specifying its joint density, executed duration, reward aggregation, and discount interval.

Bibliography

Abbas Abdolmaleki, Bilal Piot, Bobak Shahriari, Jost Tobias Springenberg, Tim Hertweck, Rishabh Joshi, Junhyuk Oh, Michael Bloesch, Thomas Lampe, Nicolas Heess, Jonas Buchli, and Martin Riedmiller. Preference optimization as probabilistic inference. *arXiv*, 2410.04166v1, 2024. URL https://arxiv.org/abs/2410.04166v1.

Zengjue Chen, Peidong Liu, Jiawei Li, and Qi Wang. HaWMPO: Hallucination-aware world model-based policy optimization for generalist robot policy. *arXiv*, 2609.09941v1, 2026. URL https://arxiv.org/abs/2609.09941v1.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024. URL https://arxiv.org/abs/2303.04137v5. Online journal edition identified by the authors as 2024; inspected arXiv:2303.04137v5. Earlier RSS conference version appeared in 2023.

Haoyi Duan, Liliang Chen, Nan Wang, Yang Zhang, Yuhang Lu, Xiaotong Chen, Wei-Cheng Tseng, Shenyuan Gao, Zhaoshuo Li, Jingyi Jin, Pooya Jannaty, Dhruv Shah, Yuke Zhu, Ming-Yu Liu, Maoqing Yao, and Guanghui Ren. WorldOracle: Benchmarking action-conditioned world models against real-world executions. *arXiv*, 2610.18826v4, 2026. URL https://arxiv.org/abs/2610.18826v4.

Yanjiang Guo, Tony Lee, Lucy Xiaoyang Shi, Jianyu Chen, Percy Liang, and Chelsea Finn. VLAW: Iterative co-improvement of vision-language-action policy and world model. *arXiv*, 2602.12063v2, 2026a. URL https://arxiv.org/abs/2602.12063v2.

Yanjiang Guo, Lucy Xiaoyang Shi, Jianyu Chen, and Chelsea Finn. Ctrl-World: A controllable generative world model for robot manipulation. *arXiv*, 2510.10125v3, 2026b. URL https://arxiv.org/abs/2510.10125v3.

Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv*, 2509.24527v1, 2025. URL https://arxiv.org/abs/2509.24527v1.

Arnav Kumar Jain, Yilin Wu, Jesse Farebrother, Gokul Swamy, and Andrea Bajcsy. WEAVER, better, faster, longer: An effective world model for robotic manipulation. *arXiv*, 2606.13672v2, 2026. URL https://arxiv.org/abs/2606.13672v2.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://papers.neurips.cc/paper/9416-when-to-trust-your-model-model-based-policy-optimization.pdf.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. URL https://arxiv.org/abs/2308.04079.

Diederik P. Kingma and Ruiqi Gao. Understanding diffusion objectives as the ELBO with simple data augmentation. *arXiv*, 2303.00848v7, 2023. URL https://arxiv.org/abs/2303.00848v7.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=PqvMRDCJT9t. Also consulted arXiv:2210.02747v2.

Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, and Itai Gat. Flow matching guide and code. *arXiv*, 2412.06264v1, 2024. URL https://arxiv.org/abs/2412.06264v1.

Qi Lv, Jiayi Lin, Yuxin Jiang, Jianzong Wang, Xiang Deng, and Mike Shou Zheng. Skeleton-grounded latent action model for robot world models. *arXiv*, 2610.31474v1, 2026. URL https://arxiv.org/abs/2610.31474v1.

David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients. *arXiv*, 2507.21053v2, 2025a. URL https://arxiv.org/abs/2507.21053v2.

David McAllister et al. Flow matching policy gradients: Authors' explanation, 2025b. URL https://flowreinforce.github.io/. Accessed 27 October 2026.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, 2020. URL https://www.matthewtancik.com/nerf.

MuJoCo Developers. Computation: General framework and soft contact model, 2026. URL https://mujoco.readthedocs.io/en/stable/computation/index.html. Online documentation accessed 27 October 2026; year identifies the accessed edition.

OpenAI. Spinning up: Introduction to policy optimization, 2018a. URL https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html. Accessed 27 October 2026.

OpenAI. Spinning up: Proximal policy optimization, 2018b. URL https://spinningup.openai.com/en/latest/algorithms/ppo.html. Accessed 27 October 2026.

OpenAI. Spinning up: Key concepts in RL, 2018c. URL https://spinningup.openai.com/en/latest/spinningup/rl_intro.html. Accessed 27 October 2026.

Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. MIT Press, 4 edition, 2023. URL https://www.pbr-book.org/4ed/Introduction.

Physical Intelligence. π_0: A vision-language-action flow model for general robot control. *arXiv*, 2410.24164v1, 2024. URL https://arxiv.org/abs/2410.24164v1.

Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv*, 2504.16054v1, 2025a. URL https://arxiv.org/abs/2504.16054v1.

Physical Intelligence. $\pi^*_{0.6}$: A VLA that learns from experience. *arXiv*, 2511.14759v1, 2025b. URL https://arxiv.org/abs/2511.14759v1.

Cheng Qian, Ruomeng Fan, Yifei Ren, Yilong Wang, and Edward Johns. SynthICL: Scalable in-context imitation learning with synthetic data. *arXiv*, 2606.08154v1, 2026a. URL https://arxiv.org/abs/2606.08154v1.

Cheng Qian, Ruomeng Fan, Yifei Ren, Yilong Wang, and Edward Johns. SynthICL: Official project page, 2026b. URL https://synth-icl.github.io/. Accessed 27 October 2026.

Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, 2011. URL https://proceedings.mlr.press/v15/ross11a.html.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. URL https://arxiv.org/abs/1506.02438v6. Inspected arXiv:1506.02438v6, revised 2018.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv*, 1707.06347v2, 2017. URL https://arxiv.org/abs/1707.06347v2.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. URL http://incompleteideas.net/book/RLbook2020.pdf. Author-hosted 2020 PDF; Sections 3.5 and 6.1 consulted.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html. Also consulted arXiv:1706.03762v7, revised 2023.

Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2411.12633v2. Inspected arXiv:2411.12633v2.

Caoliwen Wang, Mengdi Wang, Heng Zhang, Shixun Huang, Siyuan Chen, Chao Liu, Anpei Chen, Zhendong Wang, Peter Yichen Chen, and Huamin Wang. WorldContact: A contact-centric world model for scalable robot learning. *arXiv*, 2609.19600v1, 2026a. URL https://arxiv.org/abs/2609.19600v1.

Yixuan Wang, Rhythm Syed, Fangyu Wu, Mengchao Zhang, Aykut Onol, Jose Barreiros, Hooshang Nayyeri, Tony Dear, Huan Zhang, and Yunzhu Li. Interactive world simulator for robot policy training and evaluation. *arXiv*, 2603.08546v1, 2026b. URL https://arxiv.org/abs/2603.08546v1.

Jiazhi Yang, Kunyang Lin, Jinwei Li, Wencong Zhang, Tianwei Lin, Longyan Wu, Zhizhong Su, Hao Zhao, Ya-Qin Zhang, Li Chen, Ping Luo, Xiangyu Yue, and Hongyang Li. RISE: Self-improving robot policy with compositional world model. *arXiv*, 2602.11075v2, 2026. URL https://arxiv.org/abs/2602.11075v2.

Copyright 2026, From Graphics to World Models contributors. Original text and diagrams: CC BY-NC 4.0. Code: MIT. Reproduced paper figures remain CC BY 4.0; their figure regions were extracted without changing the content or notation. Full source and license details are available in the figure credits.